

Contrato

Nombre:	manejarRespuestadeCheque (respuesta : RespuestaPag_conCheque).
Responsabilidades:	Contestar la respuesta de autorización del servicio de autorización de cheques. Si la respuesta es aprobatoria, se concluye la venta.
Tipo o clase:	Sistema (tipo).
Referencias cruzadas:	
Notas:	<i>respuesta</i> es en realidad un registro que es necesario transformar en una <i>RespuestaAprobatoriadePagoconCheque</i> o en una <i>RespuestaReprobatoriadePagoconCheque</i> .
Salida:	
Precondiciones:	La solicitud de pago con cheque se envió al servicio de autorización de cheques.
Poscondiciones:	
■ Si la respuesta fue aprobatoria: □ Se creó una <i>aprobación RespuestaAprobatoriadePagoconCheque</i>. □ Se asoció la <i>Venta</i> a la <i>Tienda</i>, para incorporarla al registro histórico de ventas terminadas.	
■ En caso contrario, si la respuesta fue negativa: □ Se creó una <i>reprobatoria respuestaReprobatoriadePagoconCheque</i>.	

MODELADO DEL COMPORTAMIENTO EN LOS DIAGRAMAS DE ESTADO

Objetivos

- Elaborar diagramas de estado para los conceptos y los casos de uso.

33.1 Introducción

El lenguaje UML cuenta con una notación para los diagramas de estado que describen gráficamente los eventos y los estados de los objetos. En este capítulo expondremos las características más importantes de la notación, pero hay otras que no mencionaremos. Ponemos de relieve la utilización de los diagramas de estado para indicar los eventos del sistema en los casos de uso, aunque bien podrían aplicarse a cualquier tipo.

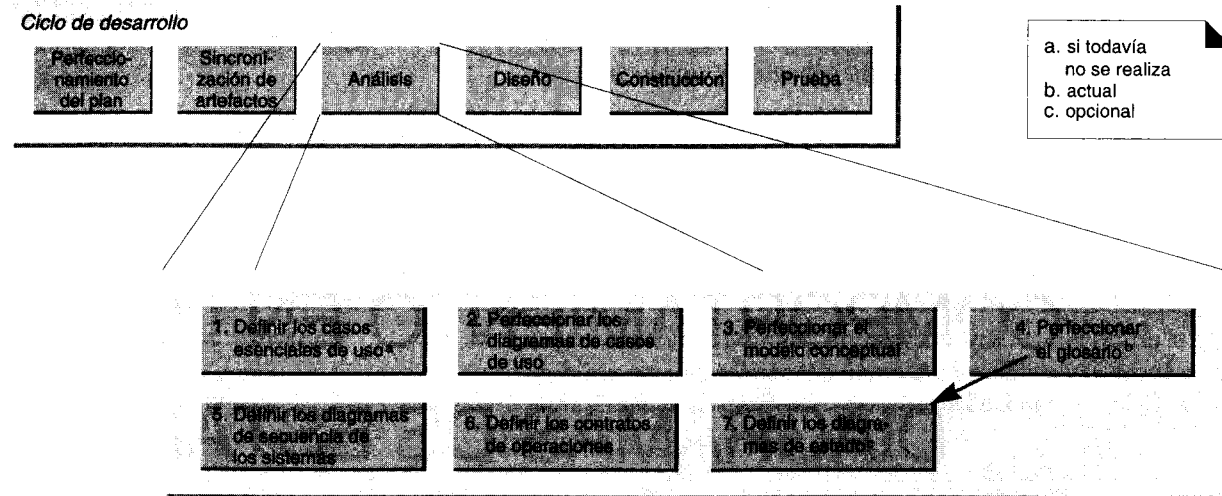
33.2 Eventos, estados y transiciones

Un **evento** es un acontecimiento importante o digno de señalar. Por ejemplo:

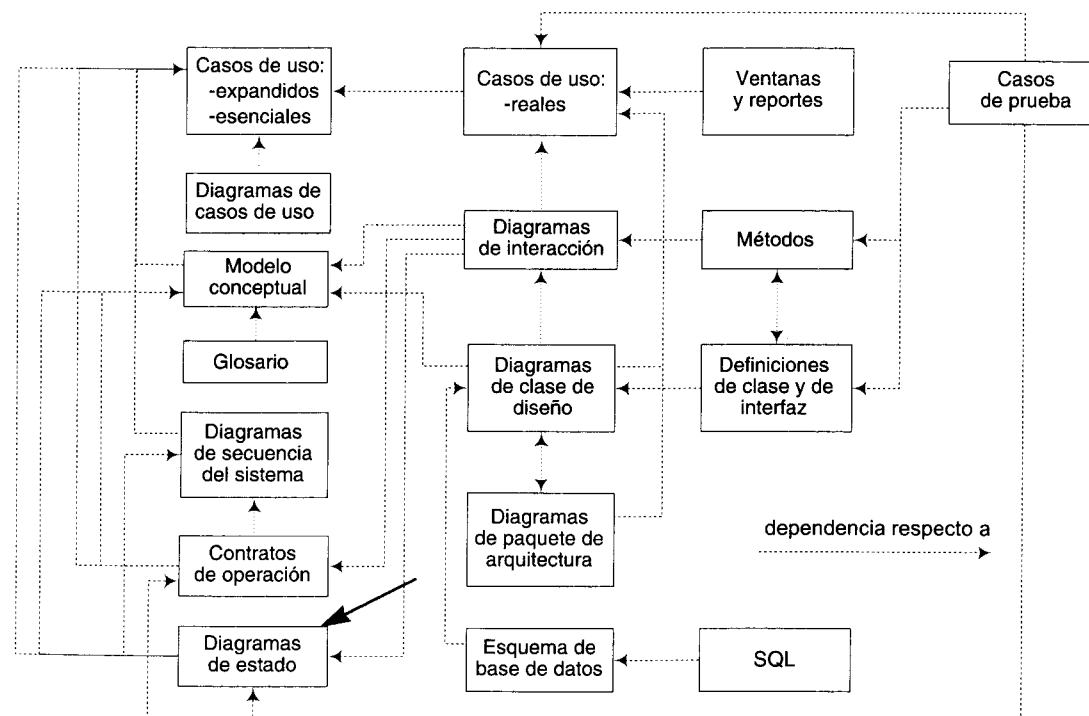
- levantar el auricular telefónico.

Un **estado** es la condición de un objeto en un momento determinado: el tiempo que transcurre entre eventos. Por ejemplo:

- un teléfono se encuentra en estado “ocioso” una vez que el auricular es puesto en su sitio y mientras no lo levantemos.



Actividades de la fase de análisis dentro de un ciclo de desarrollo.



Dependencias de los artefactos durante la fase de construcción.

La **transición** es una relación entre dos estados; indica que, cuando ocurre un evento, el objeto pasa del estado anterior al siguiente. Por ejemplo:

- cuando ocurre el evento "levantar el auricular", el teléfono realiza la transición del estado "ocioso" al estado "activo".

33.3 Diagramas de estado

Un diagrama de estado del UML (figura 33.1) describe visualmente los estados y eventos más interesantes de un objeto, así como su comportamiento ante un evento. Las transiciones se muestran con flechas que llevan el nombre del evento respectivo. Los estados se colocan en óvalos. Se acostumbra incluir un seudoestado inicial que cumple automáticamente la transición a otro estado en el momento de crear una instancia.

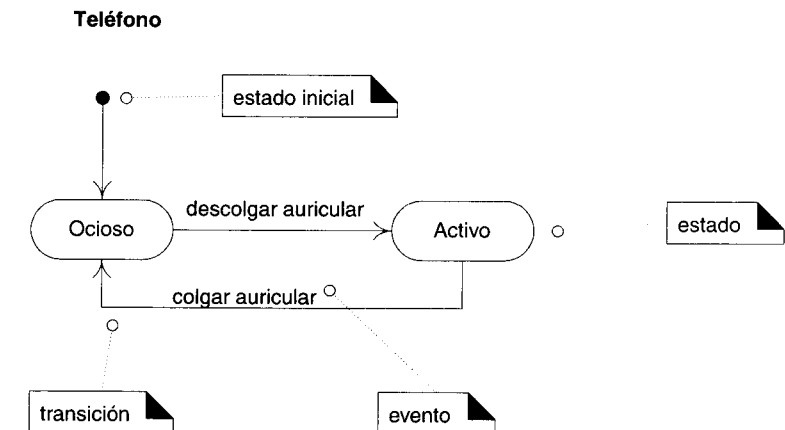


Figura 33.1 Diagrama de estado para un teléfono.

Un diagrama de estado presenta el ciclo de vida de un objeto: los eventos que le ocurren, sus transiciones y los estados que median entre esos eventos. No es necesario que muestre todos los eventos posibles; si tiene lugar un evento que no está representado en el diagrama, se excluye siempre y cuando no sea relevante en el diagrama de estado. Podemos, pues, crear un diagrama que describa el ciclo de vida de un objeto en niveles arbitrariamente simples o complejos, según las necesidades del momento.

33.3.1 Áreas de un diagrama de estado

Un diagrama de estado puede aplicarse a varios elementos del UML, a saber:

- clases de software
- tipos (conceptos)
- casos de uso

Podemos representar un “sistema” entero con un tipo, concepto o conjunto de sistemas en un dominio del problema que abarque los sistemas distribuidos; de ahí la posibilidad de que cuente con su propio diagrama de estado.

33.4 Diagramas de estado para los casos de uso

Una aplicación útil de este tipo de diagramas consiste en describir la secuencia permitida de los eventos externos del sistema que reconoce y maneja un sistema dentro del contexto de un caso de uso. Por ejemplo:

- En el caso *Comprar Productos* de la aplicación del punto de venta, no está permitido efectuar la operación *efectuarPagoconTarjeta* mientras no haya ocurrido el evento *terminarVenta*.
- En el caso *Procesar Documento* con un procesador de palabras, no está permitido efectuar la operación Guardar en Archivo mientras no haya ocurrido el evento Archivo-Nuevo o Archivo-Abierto.

Un diagrama de estado que describe los eventos globales del sistema y su secuencia en un caso de uso es una clase de **diagrama de estado para casos de uso**. El diagrama de la figura 33.2 constituye una versión simplificada de los eventos del sistema del caso de uso *Comprar Productos* en la aplicación punto de venta. Indica que no es correcto generar un evento *efectuarPago* si el evento *terminarVenta* no ha hecho que el sistema realice la transición al estado *EnEsperadelPago*.

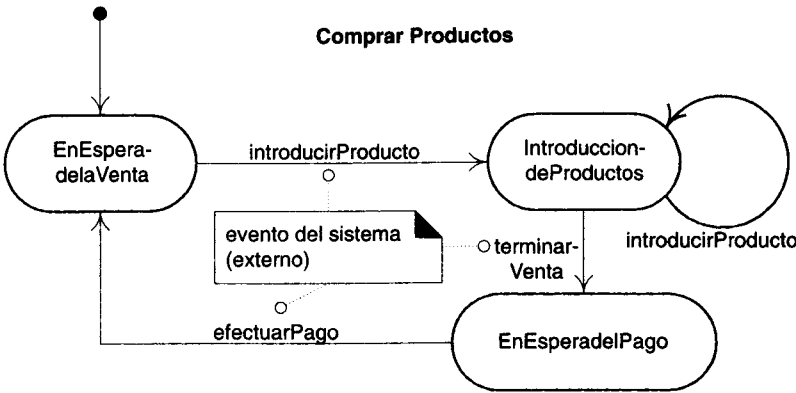


Figura 33.2 Diagrama de estado del caso de uso *Comprar Productos*.

33.4.1 Utilidad de los diagramas de estado para los casos de uso

El número de eventos de un sistema y su orden correcto en el caso *Comprar productos* tienen (hasta hoy) apariencia trivial; por eso, no parece obligatorio emplear un diagrama de estado para señalar la secuencia correcta. Pero en un caso complejo con multitud de eventos de sistema —cuando se utiliza un procesador de palabras, por ejemplo— conviene recurrir a un diagrama de estado que describa el orden legal de los eventos externos.

He aquí como hacerlo: durante las fases de diseño e implementación, hay que preparar e implementar un diseño que garantice que no ocurran eventos fuera de la secuencia, pues de lo contrario podría sobrevenir una condición de error. Por ejemplo, no debe permitirse que TPDV reciba un pago cuando aún no se concluye una venta; habrá que escribir un código que garantice esto.

Con un conjunto de diagramas de estado para los casos de uso, el diseñador podrá desarrollar metódicamente un diseño que garantice el orden correcto de eventos del sistema. A continuación se enumeran algunas soluciones posibles del diseño:

- pruebas condicionales rigurosamente codificadas para los eventos fuera de orden
- uso del patrón *Estado* que se explica en un capítulo posterior
- inactivación de elementos (widgets) en las ventanas activas para prohibir los eventos ilegales (método recomendable)
- un intérprete de una máquina de estados, el cual ejecute una tabla de estados que represente un diagrama de estado para los casos de uso

En un dominio con muchos eventos del sistema, la concisión y el rigor al emplear los diagramas de estado de casos de uso le ayudan al diseñador a cerciorarse de que no omite nada.

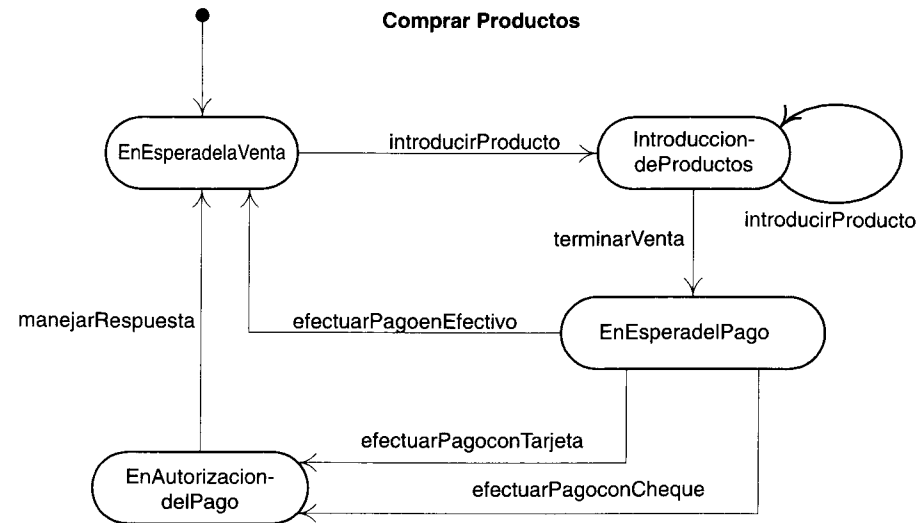
33.5 Diagramas de estado del sistema

Una variante de este tipo de diagramas es el **diagrama de estado del sistema**, que muestra las transiciones de los eventos del sistema a lo largo de todos los casos de uso. Es la unión de todos los diagramas de estado y resulta útil mientras el número total de los eventos sistémicos sea lo bastante pequeño para que sigan siendo manejables.

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

33.6 Diagramas de estado de los casos de uso para la aplicación del punto de venta

33.6.1 Comprar Productos



33.6.2 Iniciar

El diagrama de estado para el caso de uso *Iniciar* no es interesante y por ello no lo incluimos. En la práctica, este tipo de diagramas no es necesario si no hay un ordenamiento significativo de los eventos del sistema.

33.7 Tipos que requieren diagramas de estado

Además de los diagramas de estado para los casos de uso o el sistema global, podemos crear diagramas prácticamente para cualquier tipo o clase.

33.7.1 Tipos independientes y dependientes del estado

Si un objeto siempre reacciona igual ante un evento, se le considera **independiente del estado** (o sin modo) respecto a dicho evento. Por ejemplo, si un objeto recibe un mensaje y si su método de respuesta siempre hace lo mismo, el método generalmente no tendrá una lógica condicional. El objeto es independiente del estado respecto a ese mensaje. Si un tipo siempre reacciona de la misma manera ante todos los eventos de interés,

será un **tipo independiente del estado**. En cambio, los **tipos dependientes del estado** reaccionan de manera distinta ante los eventos según el estado de estos últimos.

Prepare diagramas de estado para los tipos dependientes del estado cuyo comportamiento sea complejo.

En términos generales, los sistemas de información de las empresas tienen pocos tipos interesantes dependientes del estado. Por el contrario, los dominios de control de procesos y de telecomunicaciones a menudo tienen muchos objetos dependientes del estado.

33.7.2 Tipos y clases comunes dependientes del estado

Damos a continuación una lista de clases o tipos comunes que suelen depender del estado y para los cuales posiblemente convenga elaborar un diagrama de estado:

- **Casos de uso (procesos).**
 - Visto como tipo, el caso de uso *Comprar Productos* reacciona de modo distinto ante el evento *terminarVenta* según que una venta esté realizándose o no.
- **Sistemas.** Un tipo que representa la aplicación o sistema íntegros.
 - El “*sistema del punto-de-venta*”.
- **Ventanas.**
 - La acción de Editar-Pegar sólo es válida cuando hay algo en el “portapeles” para pegar.
- **Coordinadores de aplicaciones.**
 - “Applets” en Java.
 - “Documents” en el esquema de aplicación Document-View de MFC C++ de Microsoft.
 - “ApplicationsModels” en el esquema de aplicación de Smalltalk de VisualWorks.
 - “Visual Parts” en Smalltalk de VisualAge.
- **Controladores.** Una clase que no administra aplicaciones ni ventanas y que se encarga de manejar los eventos del sistema, como se explicó en el patrón Controlador de GRASP.
 - La clase *TPDV*, que maneja los eventos *introducirProducto* y *terminarVenta* del sistema.
- **Transacciones.** La forma en que una transacción reacciona ante un evento a menudo depende de su estado actual a lo largo de todo su ciclo de vida.
 - Si una *Venta* recibió un mensaje *hacerLineadeProducto* después del evento *terminarVenta*, debería presentar una condición de error o ser omitida.

■ Dispositivos.

- TPDV, televisor, lámpara, módem; reaccionan de modo distinto ante un evento particular, según su estado actual.

■ Mutadores. Tipos que cambian su tipo o su papel.

- Una persona que cambia papeles: de civil a militar.

33.8 Otros diagramas de estado para la aplicación Punto de Venta

Los controladores (en el sentido de GRASP) son los candidatos comunes entre las sugerencias anteriores de los tipos que pueden depender del estado. Por ejemplo, el controlador que hemos usado hasta ahora en nuestra aplicación ha sido la clase *TPDV*. Nótese que esta clase, como se aprecia en la figura 33.3, tiene una prueba condicional en el método *IntroducirProducto* para determinar si se trata de una nueva venta, creando una instancia *Venta* si lo es.

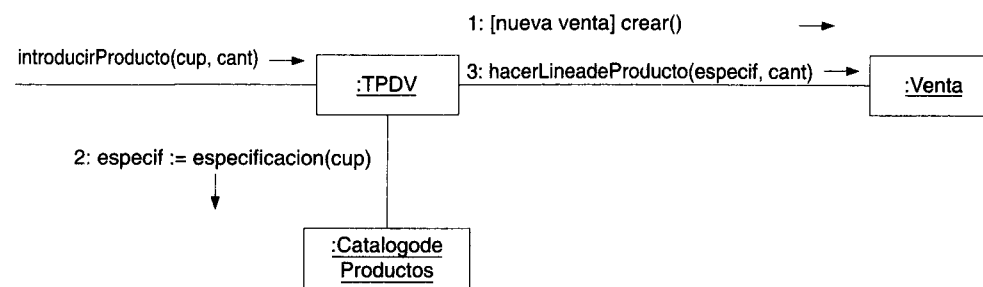


Figura 33.3 El método *introducirProducto* tiene un paso condicional, lo cual significa que TPDV depende del estado.

Una instancia *TPDV* reacciona de manera diferente ante el mensaje *IntroducirProducto* (que surge a consecuencia del evento *IntroducirProducto* del sistema), según su estado. En la figura 33.4 se incluye un diagrama de estado que explica gráficamente esto.

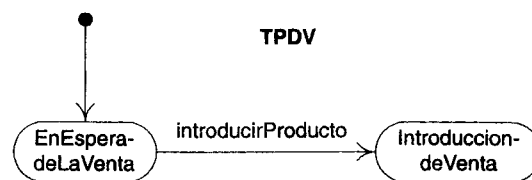


Figura 33.4 Diagrama de estado de TPDV.

33.9 Ejemplificación de eventos externos y de intervalos

33.9.1 Tipos de eventos

Conviene clasificar los eventos de la siguiente manera:

- **Evento externo:** llamado también evento del sistema, se debe a algún factor (un actor, por ejemplo) situado fuera de la frontera del sistema. Los diagramas de secuencia describen visualmente esta clase de eventos. Algunos eventos externos muy importantes provocan la invocación de las operaciones del sistema para responder a ellos.
 - Cuando un cajero oprime el botón “introducir producto” en la terminal punto de venta, significa que ha ocurrido un evento externo.
- **Evento interno:** se debe a un factor interno de nuestro sistema. Por lo que respecta al software, un evento interno tiene lugar cuando se invoca una operación a través de un mensaje o señal que partió de otro objeto interno. Los mensajes en los diagramas de colaboración indican la presencia de eventos internos.
 - Cuando una *Venta* recibe un mensaje *hacerLineadeProducto*, significa que ha ocurrido un evento interno.
- **Evento temporal:** se debe a la ocurrencia de una fecha u hora específicas o bien al transcurso del tiempo. En lo tocante al software, un reloj de tiempo real o de tiempo simulado son los que conducen este tipo de eventos.
 - Suponga que, una vez realizada una operación *terminarVenta*, debe realizarse una operación *efectuarPago* en un plazo de cinco minutos, pues de lo contrario se depurará automáticamente la venta actual.

33.9.2 Diagramas de estado para eventos internos

Un diagrama de estado puede mostrar eventos *internos* que suelen representar los mensajes recibidos de otros objetos. Los diagramas de colaboración también contienen los mensajes y sus reacciones (a partir de otros mensajes); cabe preguntar entonces: ¿por qué utilizar un diagrama de estado para describir los eventos internos y el diseño de objetos? El enfoque del diseño orientado a objetos establece que los objetos colaboran a través de mensajes para llevar a cabo las tareas; este enfoque lo explica directamente el diagrama de colaboración de UML. Resulta un poco incongruente usar un diagrama de estado para mostrar un diseño de intercambio de mensajes y de interacción entre los objetos.¹

¹ El lector a quien le interesen publicaciones sobre el análisis y diseño orientados a objetos encontrará libros de texto y en revistas ejemplos de complejos diagramas de estado dedicados a los eventos internos y a la reacción del objeto ante ellos. En lo esencial, los diseñadores han remplazado el paradigma de la interacción y colaboración entre objetos a través de mensajes por el paradigma de objetos como máquinas de estado. Además se han servido de los diagramas de estado para diseñar el comportamiento de objetos en vez de recurrir a los diagramas de colaboración. Desde un punto de vista abstracto, ambas perspectivas son equivalentes.

Por eso tengo mis reservas y no me atrevo a recomendar, para el diseño creativo orientado a objetos, los diagramas de estado que muestren eventos internos.¹ Sin embargo, puede servirnos para resumir los resultados de un diseño, una vez que lo hayamos concluido.

En cambio, como señalamos antes al explicar los diagramas de estado para los casos de uso, un diagrama de estados para eventos *externos* puede ser una herramienta sucinta y de mucha utilidad.

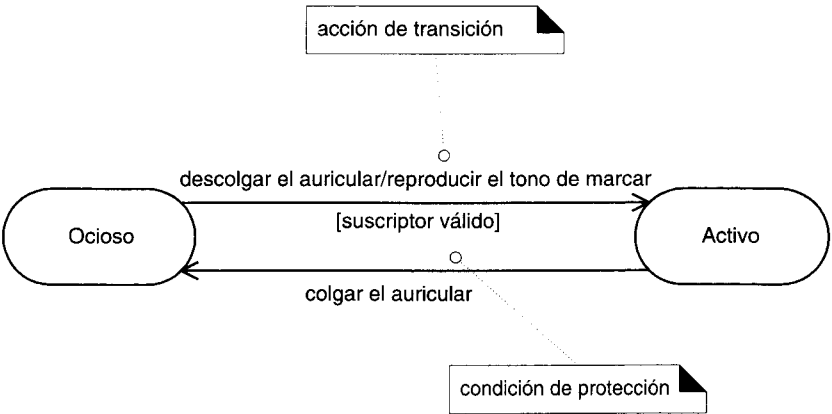
Prefiera los diagramas de estado para describir eventos externos y temporales, así como su reacción frente a ellos, en vez de servirse de ellos para diseñar el comportamiento de los objetos a partir de eventos internos.

33.10 Notación complementaria de los diagramas de estado

La notación de los diagramas de estado en el lenguaje UML contiene muchas características que no utilizaremos en esta introducción. Tres de las más importantes son:

- acciones de transición
- condiciones protectoras de las transiciones
- estados anidados

33.10.1 Acciones y protecciones de las transiciones

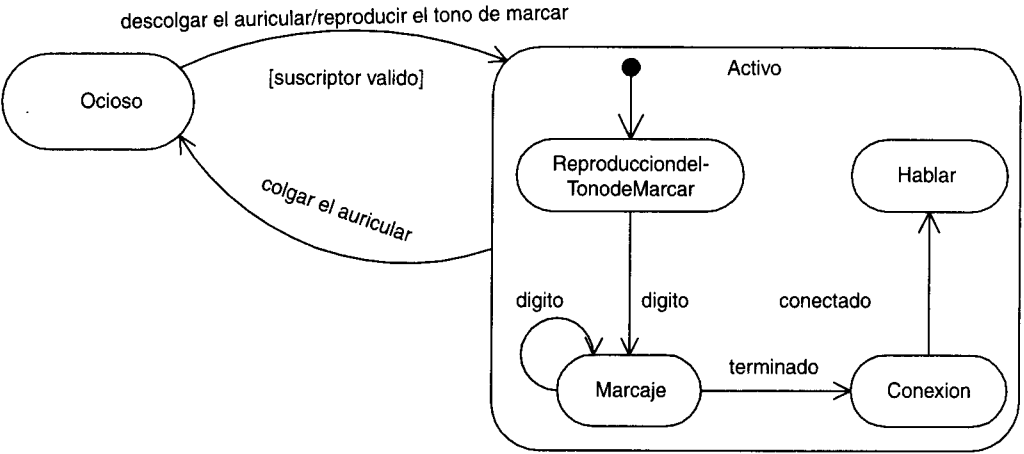


Una transición puede hacer que una acción se active. En una implementación de software, esto puede representar la invocación de un método de clases del diagrama de estado.

¹ Los diagramas de estado sirven, entre otras cosas, para mostrar el diseño del objeto basándose en los eventos internos; esta aplicación es razonable cuando hay que producir un código con un generador de código que se basará en el diagrama de estado o cuando se use un intérprete de la máquina de estado para que ejecute el software.

Una transición también puede tener una protección condicional, o prueba booleana. Sólo la tomamos si pasa la prueba.

33.10.2 Estados anidados



Un estado permite que la anidación contenga subestados; un subestado hereda las transiciones de su superestado (el estado incluyente). Se trata de una contribución fundamental de la notación de Harel para los diagramas de estado en que se funda el lenguaje UML, pues da origen a diagramas sucintos. Los subestados pueden describirse gráficamente anidándolos en una casilla o rectángulo que representa al de superestado.

Por ejemplo, cuando ocurre una transición al estado *Activo*, también se realizan la creación y la transición al subestado *ReproducciondelTonodeMarcar*. Sin importar el subestado en que se encuentre el objeto, si ocurre el evento *colgar el auricular* que esté relacionado con el superestado *Activo*, se efectuará la transición al estado *Ocioso*.

PARTE VII **FASE**
DE DISEÑO (2)

GRASP: MÁS PATRONES PARA ASIGNAR RESPONSABILIDADES

Objetivos

- Aprender a aplicar el resto de los patrones GRASP.

34.1 GRASP: Patrones generales de software para asignar responsabilidades

En páginas anteriores estudiamos la aplicación del primero de los cinco patrones GRASP:

- Experto, Creador, Alta Cohesión, Bajo Acoplamiento, Controlador

Los últimos cuatro patrones GRASP son:

- Polimorfismo
- Fabricación Pura
- Indirección
- No Hables con Extraños

En este capítulo examinaremos el resto de los patrones, los cuales constituyen los principios fundamentales con que se asignan responsabilidades a los objetos. En el siguiente capítulo trataremos de otros patrones de mucha utilidad y luego los aplicaremos al desarrollo de la segunda iteración en la aplicación terminal punto de venta.

34.2 Polimorfismo

Solución Cuando por el tipo varían las alternativas o comportamientos afines, las responsabilidades del comportamiento se asignarán —mediante operaciones polimórficas— a los tipos en que el comportamiento presenta variantes.¹

Corolario: no realizar pruebas con el tipo de un objeto ni utilizar lógica condicional para plantear diversas alternativas basadas en el tipo.

Problema ¿Cómo manejar las alternativas basadas en el tipo? ¿De qué manera crear componentes de software conectables?

Alternativas basadas en el tipo. La variación condicional es un tema esencial en la programación. Si se diseña un programa mediante la lógica condicional if-then-else o con el estatuto case, habrá que modificar la lógica del case cuando surja una variante. Este procedimiento dificulta extender un programa con otras variantes, porque los cambios tienden a requerirse en varios lugares donde exista la lógica condicional.

Componentes de software conectables. Viendo los componentes en las relaciones cliente-servidor, ¿cómo podemos remplazar un componente servidor sin afectar al cliente?

Ejemplo En la aplicación del punto de venta, ¿quién debería encargarse de autorizar las diversas clases de pagos?

El comportamiento de autorización varía con el tipo de pago —en efectivo,² con tarjeta de crédito o con cheque—; por eso, conforme al polimorfismo deberíamos asignar esa responsabilidad a cada tipo de pago, implementado con una operación polimórfica *autorizar* (figura 34.1). Cada una de estas operaciones se *instrumentará* de modo diferente: *PagosconTarjeta* se comunicará con un servicio de autorización de crédito y así sucesivamente.

¹ El **polimorfismo** tiene varios sentidos. Dentro de este contexto significa “asignar el mismo nombre a servicios en varios objetos” [Coad95], cuando los servicios se parecen o están relacionados entre ellos. Los tipos de objetos suelen estar relacionados en una jerarquía con una superclase común, aunque ello no sea estrictamente necesario (sobre todo en lenguajes de ligado dinámico, como Smalltalk, o en los que dan soporte a interfaces, como Java).

² Algunas terminales situadas en el punto de venta están provistas de un dispositivo que autentifica los billetes, pues determinan si son falsos o no. Por ejemplo, ésta es una práctica muy generalizada en algunos países europeos.

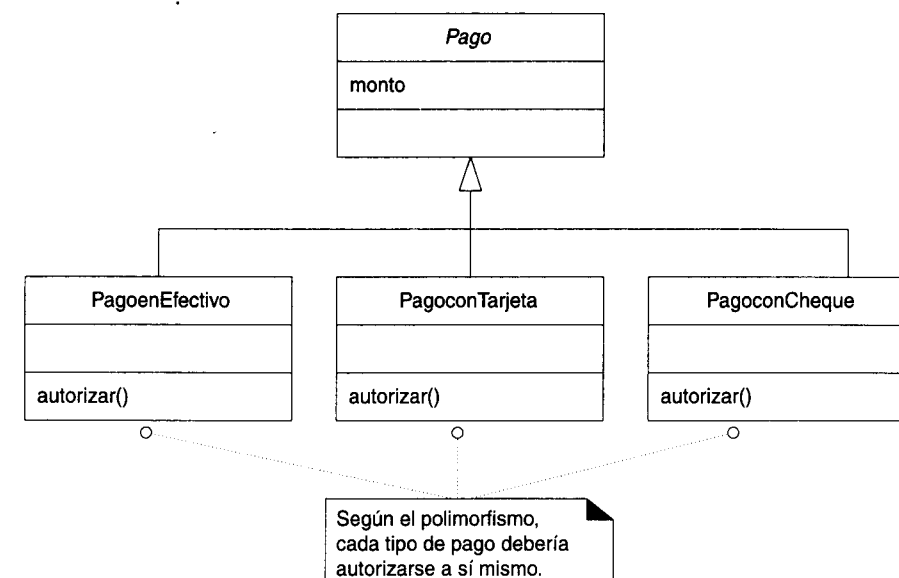


Figura 34.1 Polimorfismo en la autorización de pagos.

Explicación Como el patrón Experto, el uso del patrón Polimorfismo está acorde al espíritu del patrón “Lo hago yo mismo” [Coad95]. En vez de operar externamente sobre un pago para autorizarlo, el pago se autoriza a sí mismo; esto constituye la esencia de la orientación a objetos.

Si podemos caracterizar Experto como el patrón fundamental *táctico*, Polimorfismo será el más importante patrón *estratégico* en el diseño orientado a objetos. Es un principio fundamental en que se fundan las estrategias globales, o planes de ataque, al diseñar cómo organizar un sistema para que se encargue del trabajo. Un diseño basado en la asignación de responsabilidades mediante el polimorfismo puede ser extendido fácilmente para que realice nuevas variantes. Por ejemplo, agregar una nueva clase *PagoconTarjetaDebito* su operación polimórfica *autorizar* afectará poco el diseño actual dada la forma como se maneja la autorización.

Cuando vemos los objetos en las relaciones de cliente-servidor, los objetos del cliente requieren poca o nula modificación al introducir un nuevo objeto servidor, a condición de que el nuevo servidor soporte las operaciones polimórficas que espera el cliente.

Beneficios ■ Es fácil agregar las futuras extensiones que requieren las variaciones imprevistas.

También conocido como o semejante a “Lo hago yo mismo”, “Elección de mensaje”, “No preguntes ¿qué clase es?”

34.3 Fabricación Pura

Solución Asignar un conjunto altamente cohesivo de responsabilidades a una clase artificial que no representa nada en el dominio del problema: una cosa inventada para dar soporte a una alta cohesión, un bajo acoplamiento y reutilización.

Esa clase es una *fabricación* de la imaginación. En teoría, las responsabilidades que se asignan brindan soporte a una alta cohesión y a bajo acoplamiento, de modo que el diseño de la fabricación sea muy limpio, o *puro*. De ahí el nombre: fabricación pura.

Finalmente, una fabricación pura supone exige algo, y esto lo hacemos cuando estamos desesperados.

Problema ¿A quién asignar la responsabilidad cuando uno está desesperado y no quiere violar los patrones Alta Cohesión y Bajo Acoplamiento?

Los diseños orientados a objetos se caracterizan por implementar como clases de software las representaciones de conceptos en el dominio de un problema del mundo real; por ejemplo, una clase *Venta* y *Cliente*. Pese a ello, se dan muchas situaciones donde el asignar responsabilidades exclusivamente a las clases del dominio origina problemas por una mala cohesión o acoplamiento o bien por un escaso potencial de reutilización.

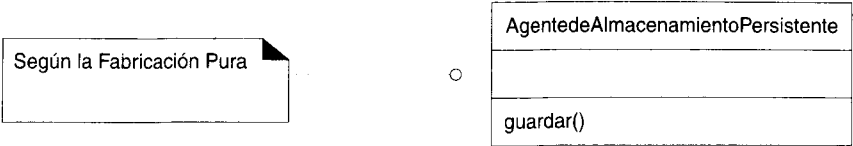
Ejemplo Supongamos, por ejemplo, que se necesita soporte para guardar las instancias *Venta* en una base de datos relacional. En virtud del patrón Experto, en cierto modo se justifica asignar esta responsabilidad a la clase *Venta*. Pero reflexionemos sobre las siguientes implicaciones:

- La tarea requiere un número relativamente amplio de operaciones de soporte orientadas a la base de datos, ninguna de las cuales se relaciona con el concepto de vender; por tanto, la clase *Venta* reduce su cohesión.
- La clase *Venta* ha de ser acoplada a la interfaz de la base de datos relacional (interfaz que suele proporcionar el proveedor de las herramientas de desarrollo); de ahí que mejore su acoplamiento. Y éste ni siquiera se realiza con otro objeto del dominio, sino con una interfaz idiosincrásica de la base de datos.
- Guardar los objetos en una base de datos relacional es una tarea muy general en que debemos brindar soporte a muchas clases. Asignar estas responsabilidades a la clase *Venta* indica que habrá poca reutilización o mucha duplicación en otras clases que cumplen la misma función.

En conclusión, aunque en virtud del patrón Experto *Venta* es un candidato lógico para guardarse a sí misma en una base de datos, da origen a un diseño de baja cohesión, alto acoplamiento y bajo potencial de reutilización, precisamente el tipo de situación desesperada que exige inventar algo.

Una solución razonable consiste en crear una clase nueva que se encargue tan sólo de guardar los objetos en algún tipo de almacenamiento persistente: una base de datos

relacional; lo llamaremos *AgentedeAlmacenamientoPersistente*.¹ Esta clase es una Fabricación Pura, una mera abstracción de la imaginación.



Sirve para resolver los siguientes problemas de diseño:

- La Venta conserva su buen diseño, con alta cohesión y bajo acoplamiento.
- La clase *AgentedeAlmacenamientoPersistente* es también relativamente cohesiva, pues su único propósito es guardar los objetos en un medio de almacenamiento persistente.
- La clase *AgentedeAlmacenamientoPersistente* es un objeto extremadamente genérico y reutilizable.

Crear una fabricación pura en este ejemplo es precisamente la situación donde se justifica su uso: hay que eliminar un diseño deficiente, con mala cohesión y acoplamiento y cambiar por un buen diseño que ofrezca un mayor potencial de reutilización.

Nótese que, igual que con los patrones GRASP, aquí se pone de relieve a quién deberán asignarse las responsabilidades. En nuestro ejemplo las transferimos de la clase *Venta* a la Fabricación Pura.

Explicación Para diseñar una Fabricación Pura debe buscarse ante todo un gran potencial de reutilización, asegurándose para ello de que sus responsabilidades sean pequeñas y cohesivas. Estas clases tienden a tener un conjunto de responsabilidades de *granularidad fina*.

Una fabricación pura suele partirse atendiendo a su funcionalidad y, por lo mismo, es una especie de objeto de función central.

Generalmente se considera que la fabricación es parte de la capa de servicios orientada a objetos de alto nivel en una arquitectura.

Muchos patrones actuales del diseño orientado a objetos constituyen ejemplos de Fabricación Pura: Adaptador, Observador, Visitante y otros más [GHJV95].

- Beneficios**
- Se brinda soporte a una Alta Cohesión porque las responsabilidades se dividen en una clase de granularidad fina que se centra exclusivamente en un conjunto muy específico de tareas afines.
 - Puede aumentar el potencial de reutilización debido a la presencia de las clases de Fabricación Pura de granularidad fina, cuyas responsabilidades pueden utilizarse en otras aplicaciones.

Problemas posibles Puede perderse el espíritu de los buenos diseños orientados a objetos que se centran en los objetos y no en las funciones, pues las clases de Fabricación Pura casi siempre

¹ En un esquema de persistencia real, definitivamente se requerirá más de una clase de Fabricación Pura para elaborar un buen diseño.

se dividen atendiendo a su funcionalidad; dicho con otras palabras, se confeccionan clases destinadas a conjuntos de funciones. Si se abusa de ello, la creación de clases de Fabricación Pura, originará un diseño centrado en procesos o funciones que se implementa en un lenguaje orientado a objetos.

- Patrones relacionados**
- Bajo Acoplamiento.
 - Alta Cohesión.
 - Una Fabricación Pura normalmente asume las responsabilidades de la clase del dominio a la cual se le asignarían basándose en el patrón Experto.
 - Muchos patrones actuales de diseño orientados a objetos son ejemplos de Fabricación Pura: Adaptador, Observador, Visitante y otros más [GHJV95].

34.4 Indirección

Solución Se asigna la responsabilidad a un objeto intermedio para que medie entre otros componentes o servicios, y éstos no terminen directamente acoplados.

El intermediario crea una *indirección* entre el resto de los componentes o servicios.

Problema ¿A quién se asignarán las responsabilidades a fin de evitar el acoplamiento directo? ¿De qué, manera se desacoplarán los objetos de modo que se obtenga un Bajo Acoplamiento y se conserve un alto potencial de reutilización?

Ejemplos **AgentedeAlmacenamientoPersistente**

El ejemplo de Fabricación Pura para desacoplar la *Venta* y los servicios de la base de datos relacional introduciendo la clase *AgentedeAlmacenamientoPersistente* es también un ejemplo de asignar responsabilidades para apoyar la Indirección. El *AgentedeAlmacenamientoPersistente* sirve de intermediario entre la *Venta* y la base de datos.

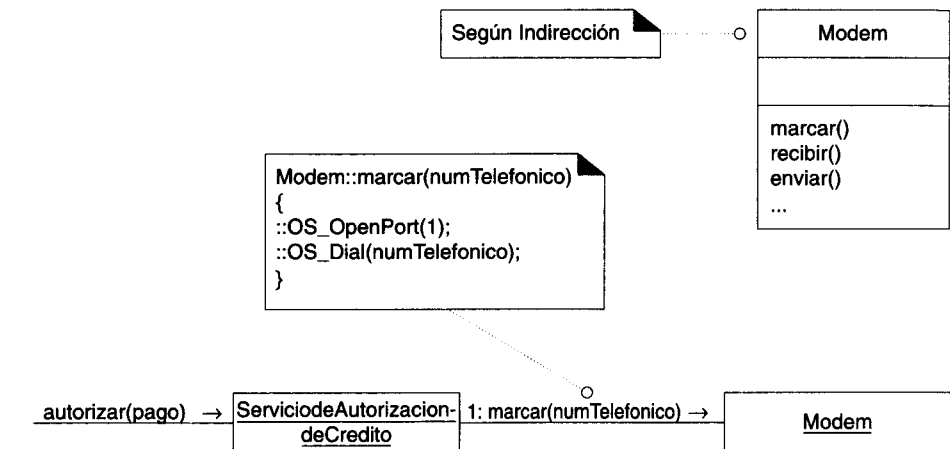
Módem

Suponga que:

- Una aplicación de terminal situada en el punto de venta necesita manipular un módem para transmitir solicitudes de pago con tarjeta de crédito.
- El sistema operativo tiene una llamada de función API de bajo nivel para hacer esto.
- Una clase denominada *ServiciodeAutorizaciondeCredito* asume la responsabilidad de hablar con el módem.

Si el *ServiciodeAutorizaciondeCredito* invoca directamente las llamadas de la función API de bajo nivel, estará estrechamente acoplado a las peculiaridades de la API del sistema operativo en cuestión. Requerirá modificación si la clase necesita ser trasladada a otro sistema operativo (para usarla en la misma aplicación o en otra).

Se agrega una clase intermedia *Modem* entre el *ServiciodeAutorizaciondeCredito* y la API del módem. Es responsable de traducir los requerimientos abstractos del módem a la API y crear una Indirección entre el *ServiciodeAutorizaciondeCredito* y la API del módem. (También se le da el nombre de agente **[proxy]** del dispositivo a una clase como *Modem* que representa a un dispositivo electromecánico y se conecta con él.)



34.4.1 Publicar-Suscribir u Observador

El patrón Publicar-Suscribir u Observador [GHJV95] también constituye un ejemplo del patrón Indirección. Los objetos se suscriben a eventos ante un *AdministradordeEventos*; otros publican eventos para el *AdministradordeEventos*, que los notifica a los suscriptores. A través de la indirección del *AdministradordeEventos* se desacoplan los editores y los suscriptores.

Explicación “La mayoría de los problemas de la computación puede resolverlos otro nivel de indirección”, reza un adagio muy acorde a los diseños orientados a objetos.¹

Del mismo modo que muchos patrones actuales de diseño son especializaciones de la Fabricación Pura, también muchos son especialización de Indirección; por ejemplo, Adaptador, Fachada y Observador [GHJV95]. Además, muchas clases de Fabricación Pura se generan a causa del patrón Indirección. El motivo de la Indirección casi siempre es el Bajo Acoplamiento; se agrega un intermediario con el fin de desacoplar otros componentes o servicios.

¹ Si es que algún adagio puede considerarse antiguo en las ciencias computacionales. No recuerdo ahora la fuente (Parnas, tal vez).

- Beneficios** ■ Bajo acoplamiento.
- Patrones** ■ Bajo Acoplamiento.
- Mediador [GHJV95].
 - Muchos intermediarios de Indirección son situaciones de Fabricación Pura.

34.5 No Hables con Extraños

Solución Se asigna la responsabilidad a un objeto directo del cliente para que colabore con un objeto indirecto, de modo que el cliente no necesite saber nada del objeto indirecto.

El patrón, también conocido con el nombre de ley de Demeter [Lieberherr88], impone restricciones a los objetos a los cuales deberíamos enviar mensajes dentro de un método. El patrón establece que en un método, los mensajes sólo deberían ser enviados a los siguientes objetos:

- 1. El objeto *this* (o *self*).
- 2. Un parámetro del método.
- 3. Un atributo de *self*.
- 4. Un elemento de una colección que sea atributo de *self*.
- 5. Un objeto creado en el interior del método.

Con esto se busca no acoplar un cliente al conocimiento de objetos indirectos ni a las representaciones internas de objetos directos. Los objetos directos son “conocidos” del cliente, los objetos indirectos son “extraños”, y un cliente debería tener sólo conocidos, no extraños.

Cumplir con las restricciones anteriores significa que los objetos directos pueden requerir nuevas operaciones que actúen como intermediarios, a fin de que el cliente pueda evitar “hablar con extraños”.

Problema ¿A quién asignar las responsabilidades para evitar conocer la estructura de los objetos indirectos?

Si un objeto conoce las conexiones internas y las estructuras de otros, entonces presentará alto acoplamiento. Si un objeto cliente tiene que usar un servicio u obtener información a partir de un objeto indirecto, ¿cómo podrá hacerlo sin acoplarse al conocimiento de la estructura interna de su servidor directo o de los objetos indirectos?

Ejemplo En una aplicación del punto de venta, suponga que una instancia *TPDV* posee un atributo referente a una *Venta*, la cual cuenta con un atributo referente a un *Pago*.

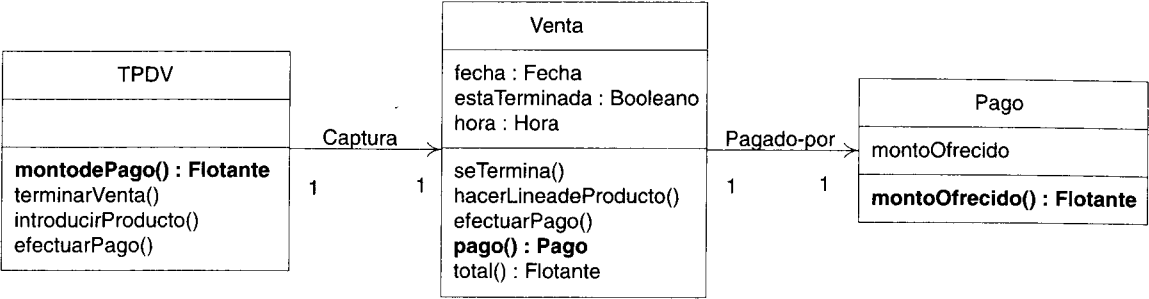


Figura 34.2 Diagrama parcial de clases del diseño.

Y además suponga que:

- Las instancias *TPDV* soportan la operación *montodePago*, que devuelve el actual monto ofrecido como pago.
- Las instancias *Venta* soportan la operación *pago*, la cual devuelve la instancia *Pago* asociada a la *Venta*.

Una forma de devolver el monto del pago es:

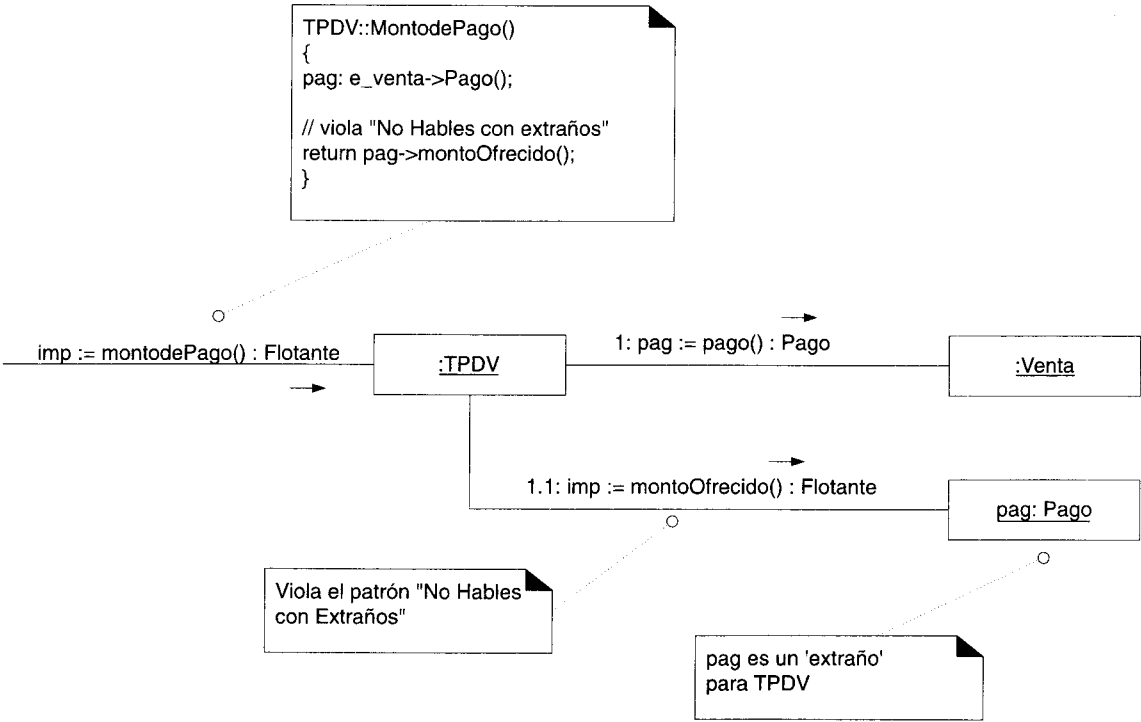


Figura 34.3 Violación del patrón “No Hables con Extraños”.

La solución de la figura 34.3 constituye una violación del principio No Hables con Extraños, porque la instancia *TPDV* envía un mensaje a un objeto indirecto: el *Pago* no es uno de los cinco candidatos “conocidos”.

La solución, como lo indica el patrón, consiste en agregar la responsabilidad al objeto directo —la *Venta*, en este caso— para que devuelva a *TPDV* el monto del pago. A esto se le llama **promoción de la interfaz**, que es la solución general que da soporte al principio general. Por tanto, se agrega una operación *montodePago*, de modo que la instancia *TPDV* no tenga que hablar con un extraño (figura 34.4).

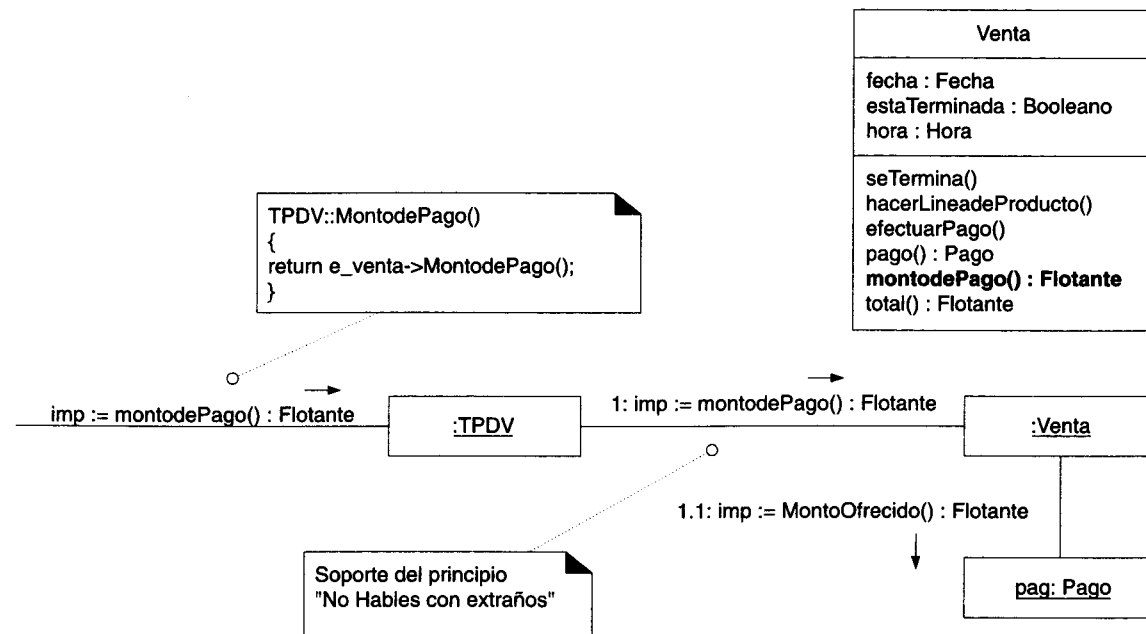


Figura 34.4 Soporte del principio “No Hables con Extraños”.

Explicación No Hables con Extraños se refiere a no obtener una visibilidad temporal frente a objetos indirectos, que son de conocimiento de otros objetos pero no del cliente. La desventaja de conseguir visibilidad ante extraños es la siguiente: la solución se acopla entonces a la estructura interna de otros objetos. Ello origina un alto acoplamiento, que hace el diseño menos robusto y más propenso a requerir un cambio si se alteran las relaciones estructurales indirectas.

Supongamos, por ejemplo, que se usa la solución de la figura 34.3 y que luego se modifica la definición de clase relativa a *Venta*, de modo que ya no conserve su referencia a *Pago*. En tal caso, el método *TPDV-montodePago* queda invalidado y requiere mantenimiento. En este ejemplo pequeño no parece existir un problema difícil, pero sí constituye un problema en un sistema con cientos —si no es que miles— de clases que son desarrolladas simultáneamente por muchos diseñadores de software. Un sistema se torna frágil cuando se diseña una solución que dependa del conocimiento de las relaciones estructurales indirectas.

En contraste con la solución de la figura 34.3, si se recurre a la de la figura 34.4, en los métodos *TPDV* no influirá ningún cambio de las representaciones ni de las conexiones internas de la *Venta*. Lo único que *TPDV* sabe es que una *Venta* puede contestar su pago total, pero sin saber cómo se consigue.

La violación de este principio es común en Smalltalk, lenguaje donde se cuenta con muchos métodos públicos para acceder a las conexiones estructurales privadas de un objeto. Un código como el siguiente aparece con mucha frecuencia:

```

miMetodoIncorrectodeObtenerPrecio
    "saltar de X a A a B a C y a D
    para regresar el precio de D"

    mixDirecta obtenerA obtenerB obtenerC obtenerD
    obtenerprecio
  
```

El código se desplaza por las conexiones estructurales saltando de un objeto al siguiente. Reflexione sobre la fragilidad del código: está estrechamente acoplado al conocimiento de muchas relaciones estructurales indirectas.

34.5.1 Violación de la ley

Lo primero que debemos conocer sobre las leyes del software es que están hechas para ser quebrantadas. Por eso, aunque la de No Hables con Extraños (ley de Demeter) parezca un excelente consejo, se dan situaciones donde conviene prescindir de ella. Una situación común donde esto ocurre se presenta cuando hay algún tipo de “agente” o “servidor de objetos” (generalmente una Fabricación Pura) encargado de devolver otros objetos, basándose para ello en una consulta mediante el valor de una clave. Se juzga aceptable obtener visibilidad ante un objeto *X* a través de un agente y luego enviarle directamente mensajes a *X*, aun cuando con ello se viole la ley No Hables con Extraños.

Beneficios ■ Bajo acoplamiento

Patrones conexos ■ Bajo Acoplamiento, Indirección, Cadena de responsabilidades [GHJV95]

DISEÑO CON MÁS PATRONES

Objetivos

- Aplicar los patrones GRASP y de la Pandilla de los Cuatro al diseñar la aplicación del punto de venta.

35.1 Introducción

En este capítulo vamos a estudiar la creación de diagramas de colaboración en la segunda iteración de la aplicación del punto de venta. Procuraremos ante todo mostrar la manera de aplicar los patrones GRASP y algunos de gran utilidad que han publicado otros autores. Demostraremos con ejemplos que el diseño orientado a objetos y la asignación de responsabilidades pueden explicarse y aprenderse utilizando los patrones: un vocabulario de principios y métodos susceptibles de combinarse para diseñar objetos.

Repetimos una vez más: la asignación de responsabilidades a los objetos y el diseño de colaboración entre ellos constituyen la esencia de este tipo de diseño. El trabajo de análisis que hemos efectuado hasta ahora se proponía dar soporte a la fase de diseño orientado a objetos.

Aunque los contratos para las operaciones de un sistema no se mencionan explícitamente aquí, las poscondiciones estipuladas en ellos se cumplen en las siguientes colaboraciones de los objetos. En la práctica, se revisan los contratos al ir desarrollando el diseño.

35.1.1 Los patrones de la Pandilla de los Cuatro

Los otros patrones que describiremos en este capítulo están tomados de *Design Patterns* [GHJV95], obra pionera en que se presentan 23 patrones de gran utilidad

durante la fase de diseño orientado a objetos. Dado que el libro fue escrito por cuatro autores, a los patrones se les conoce hoy con el nombre de “Gang of Four” (Pandilla de los Cuatro) o por la abreviatura en inglés “GoF”.¹

En este capítulo vamos a ofrecer una introducción a algunos de ellos; en capítulos subsecuentes explicaremos otros, entre ellos el Método de Plantilla (Template Method) y Observador. Recomendamos al lector estudiar detenidamente la obra *Design Patterns*.

35.2 Estado (Pandilla de los Cuatro)

Como vimos en el capítulo anterior al hablar de los diagramas de estado, la reacción de la clase *TPDV* ante el mensaje *IntroducirProducto* dependerá de su estado. Esto se explica visualmente en los diagramas de estado de las figuras 35.1 y 35.2.

En vez de servirse de una prueba condicional, un método alternativo consiste en utilizar el patrón Estado de la Pandilla de los Cuatro. En términos generales, nos sirve para eliminar las pruebas condicionales que provienen de las dependencias de estado.

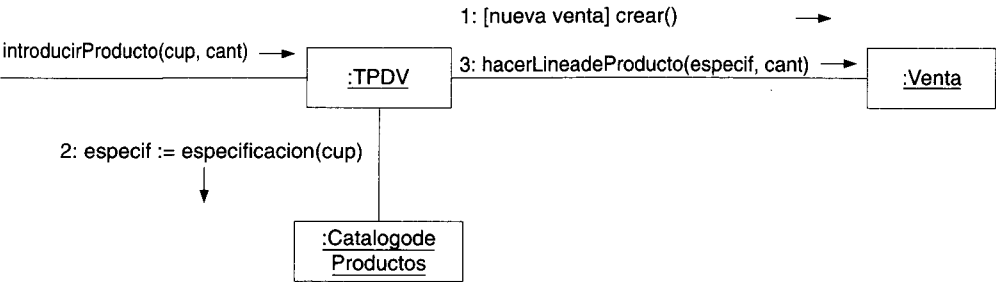


Figura 35.1 Diagrama de colaboración parcial de introducirProducto que muestra la reacción dependiente del modo.

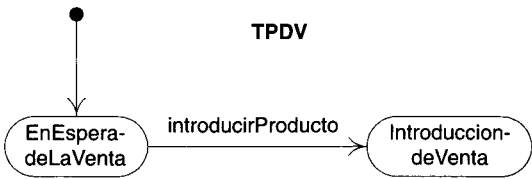


Figura 35.2 Diagrama de estado de TPDV.

¹ Con una alusión velada a la política de China.

Estado

Contexto/Problema

El comportamiento de un objeto depende de su estado. La lógica condicional no es adecuada a causa de su complejidad, escalabilidad o duplicación.

Solución

1. Crear una clase para cada estado que influya en el comportamiento del objeto dependiente del estado (el objeto “contexto”).

2. Con base en el polimorfismo, asignar métodos a cada clase de estado para manejar el comportamiento del objeto contexto.

3. Cuando el objeto contexto reciba un mensaje dependiente del estado, el mensaje será enviado al objeto estado.

Como se aprecia en la figura 35.3, se define una jerarquía de clases que representa los estados de la clase *TPDV*. Cada uno contiene un método *introducirProducto* porque su comportamiento depende del estado. Una instancia *TPDV* posee visibilidad de atributos ante su instancia actual *TPDV-Estado*.

Como se indica en la figura 35.4, cuando la instancia *TPDV* recibe el mensaje *IntroducirProducto*, lo envía a su estado y éste lo maneja mediante el patrón Polimorfismo.

Nótese que el objeto contexto se transmite junto con el objeto estado; con ello el contexto recupera la visibilidad de los parámetros y entonces puede recibir mensajes; por ejemplo, el que cambia su estado.

Comentario sobre la notación

En la figura 35.4 observe lo siguiente: se indica que el mensaje *introducirProducto* es enviado a una instancia de la superclase *TPDVEstado*, aun cuando en realidad se trata de la instancia de una de las subclases.

Para cada caso polimórfico de las subclases, se dibuja un nuevo diagrama de colaboración, comenzando con el mensaje polimórfico.

Recomendamos utilizar este estilo cuando se muestre un mensaje polimórfico.

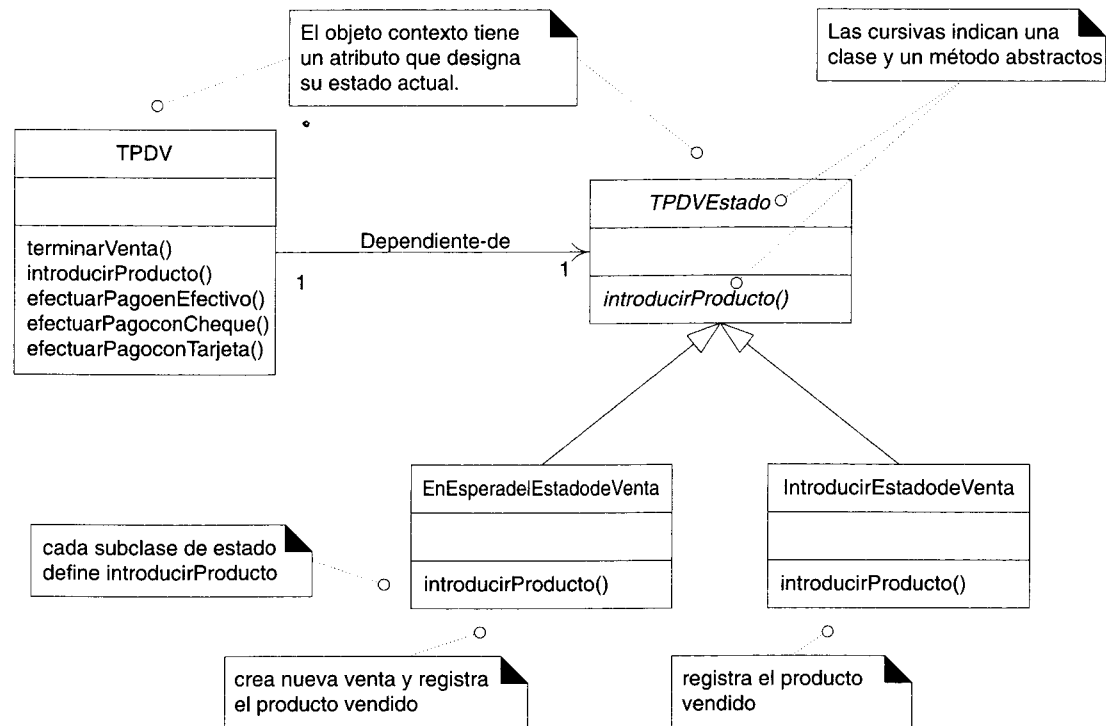


Figura 35.3 Clases de estado utilizadas en el patrón Estado.

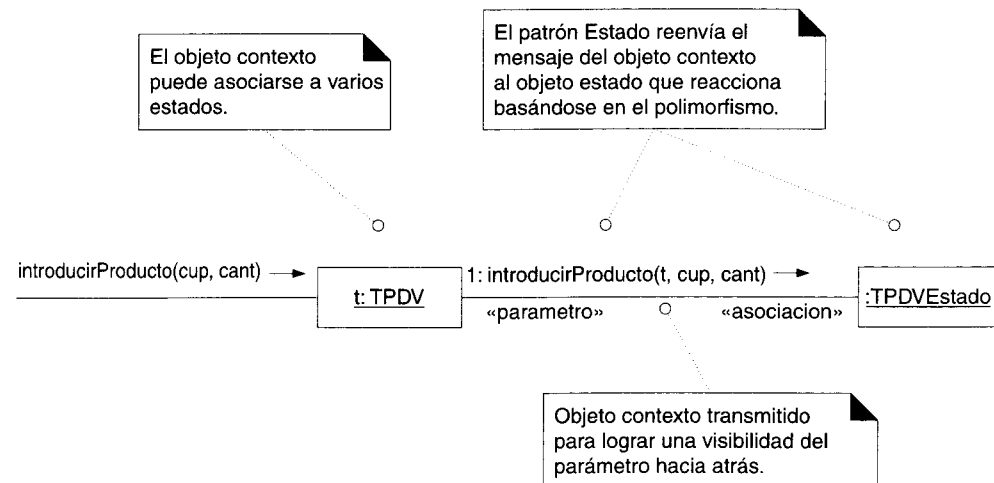


Figura 35.4 El patrón Estado comienza con el objeto contexto enviando un mensaje al objeto estado asociado.

Las figuras 35.5 y 35.6 empiezan con el mensaje *introducirProducto* enviado a una instancia de *EnEsperadelEstadodeVenta* e *IntroducirEstadodeVenta*, respectivamente.

En la figura 35.5 el método *introducirProducto* inicia la creación de una nueva venta antes de registrar el artículo vendido. Además —y esto es una parte común del patrón Estado—, el estado actual le asigna un nuevo estado al objeto contexto.

En nuestro ejemplo, se creó la instancia de un nuevo estado, pero también se acostumbra emplear el patrón **Singleton** de la Pandilla de los Cuatro, que describiremos más adelante, para asignar una instancia de estado Singleton en vez de crearla. Otro patrón afín que puede usarse con los objetos Estado es **Flyweight** de la Pandilla de los Cuatro, que se parece a Singleton. Es un patrón de gran utilidad cuando podemos compartir una instancia entre muchos objetos, porque la instancia no alberga estado alguno sino que sólo posee comportamiento. Por ejemplo, las instancias *TPDVEstado* no conservan información; el objeto contexto se transmite como parámetro cuando se necesita. En este caso, en el sistema no se requiere más que una instancia de las clases concretas (*EnEsperadelEstadodeVenta* y otras).

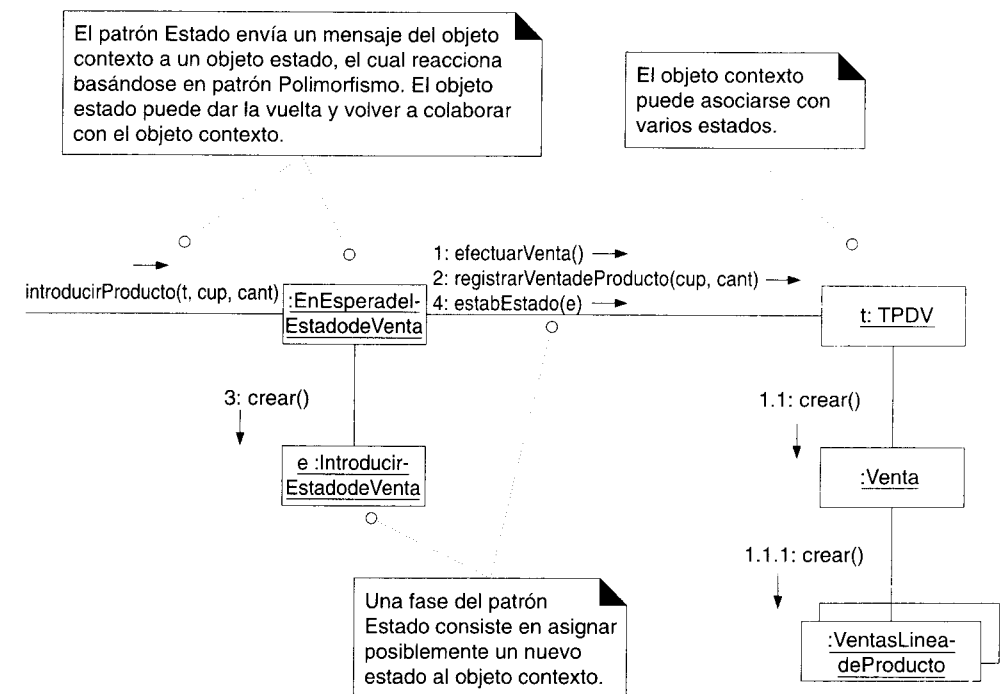


Figura 35.5 Un caso polimórfico del patrón estado: EnEsperadelEstadodeVenta.

La figura 35.6 muestra el caso polimórfico de *introducirProducto* para *IntroducirEstadodeVenta*. En este caso no es necesario crear una nueva venta, sino que basta registrar el artículo vendido. Adviértase que esta vez el estado no cambia; un método de estado no causará necesariamente un cambio de estado.

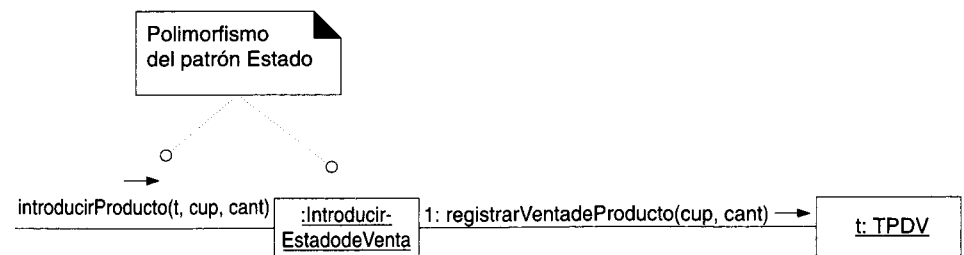


Figura 35.6 Un segundo caso polimórfico del patrón Estado: IntroducirEstadodeVenta.

Finalmente, el diagrama de colaboración registrarVentadeProductos se muestra en la figura 35.7.

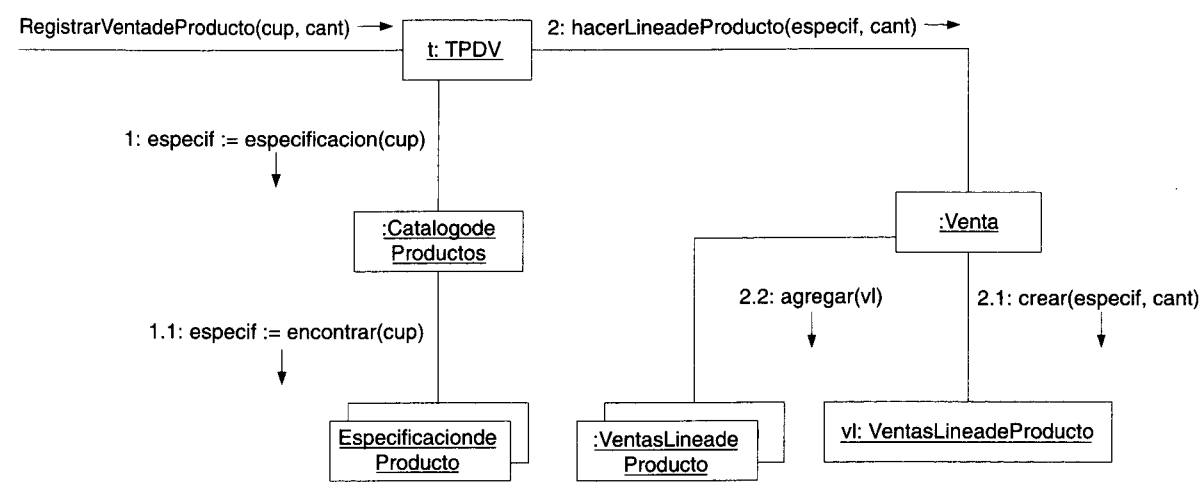


Figura 35.7 El método registrarVentadeProducto de la clase TPDV.

35.2.1 Estado: conclusión

El patrón Estado es muy útil cuando el comportamiento de un objeto depende de su estado. Con él se elimina lógica condicional en los métodos del objeto contexto y se obtiene un mecanismo elegante para extender el comportamiento de dicho objeto sin modificarlo.

Si un sistema presenta muchos estados, este patrón puede ser idóneo por la proliferación de clases. Otra alternativa consiste en definir un intérprete de la máquina de estado que funcione contra un grupo de reglas de transición de estados.

35.3 Polimorfismo (GRASP)

Como vimos en el capítulo anterior al tratar de los patrones GRASP, el hecho de que un pago se autorice a sí mismo corresponde al espíritu de los diseños orientados a objetos: el patrón “Lo hago yo mismo” [Coad95]. Además, como existen muchos tipos de pago, se requiere el patrón Polimorfismo para que cada tipo de pago se autorice a sí mismo.

Por tanto, como se observa en la figura 35.8, cada subclase *Pago* cuenta con su propio método *Autorizar*.

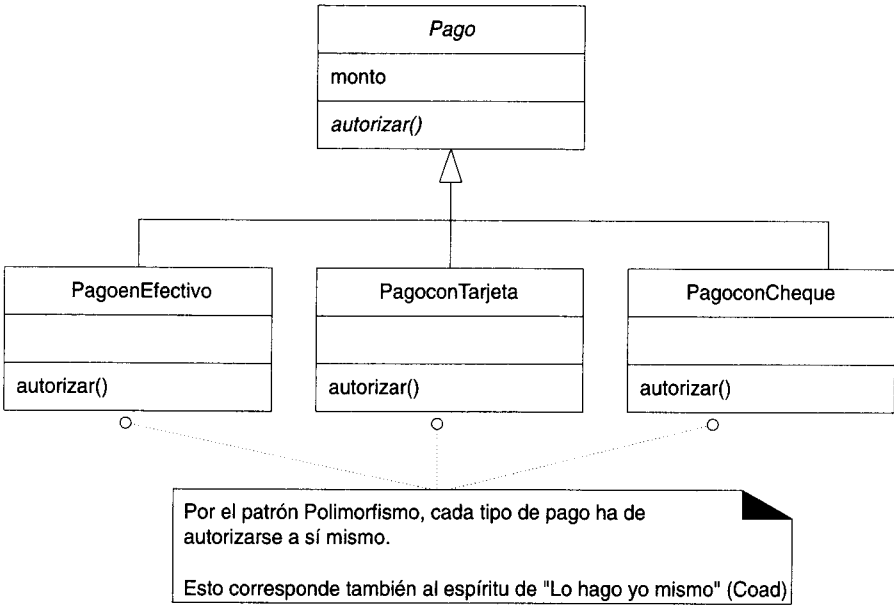


Figura 35.8 Jerarquía de pagos con múltiples métodos Autorizar.

Por ejemplo, como se muestra en las figuras 35.9 y 35.10, una *Venta* instancia un *PagoconTarjeta* o *PagoconCheque* y le pide que se autorice ella misma.

Nótese también la creación de los objetos de software *TarjetadeCredito*, *LicenciadeConducir* y *Cheque*. Un primer impulso sería registrar simplemente en sus respectivas clases de pago la información que aportan y eliminar tales clases de granularidad fina. Sin embargo, por lo regular una estrategia más provechosa consiste en usarlas; a menudo terminan ofreciendo un comportamiento de gran utilidad y siendo muy reutilizables. Por ejemplo, *TarjetadeCredito* es un Experto natural al indicarnos su tipo de institución de crédito (Visa, MasterCard y otros más). Este comportamiento resultará necesario en nuestra aplicación.

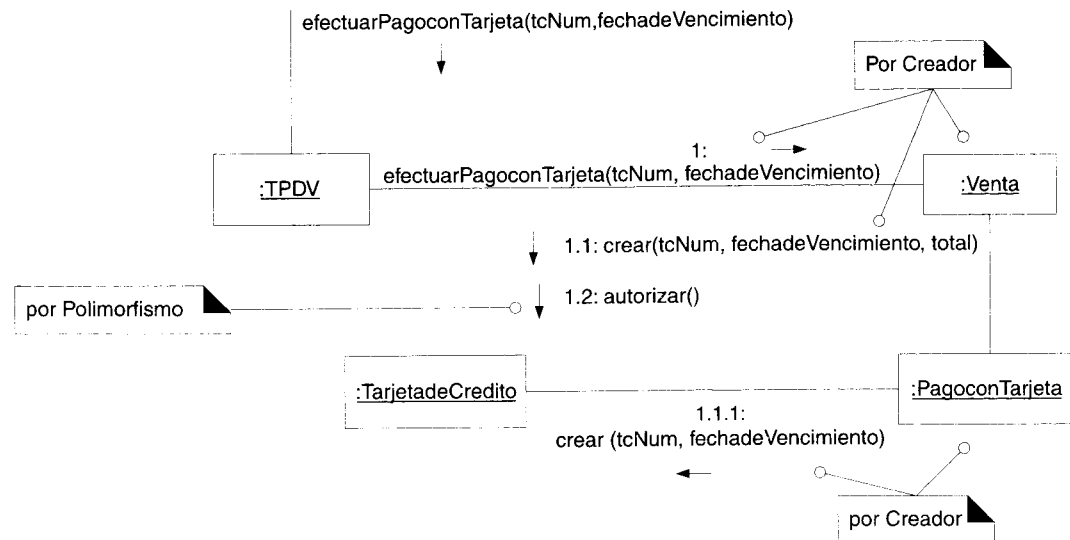


Figura 35.9 Creación de un PagoconTarjeta.

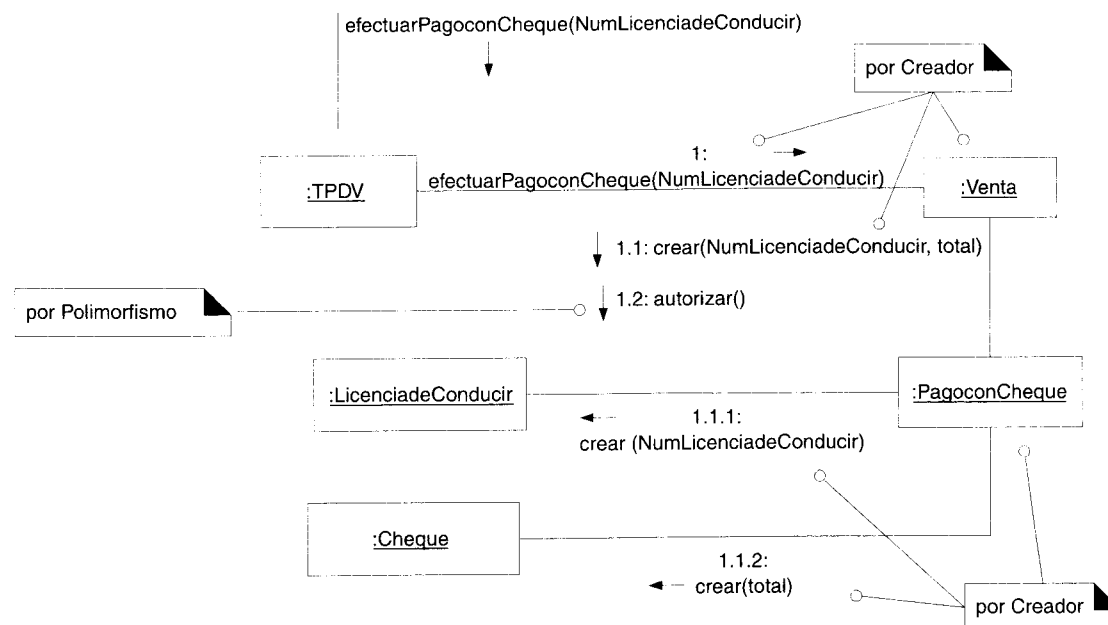


Figura 35.10 Creación de un PagoconCheque.

35.4 Singleton

Cuando un *PagoconTarjeta* recibe un mensaje *Autorizar*, necesita enviar un mensaje a la *Tienda* para determinar con cuál servicio de autorización de crédito se comunicará (Bank of Foo para Visa, Bank of Bar para MasterCard, etc.). ¿Por qué preguntarle a *Tienda*? Porque es un Experto natural en la información concerniente a los servicios de autorización.

Pero surge aquí un problema de visibilidad: la instancia recién creada *PagoconTarjeta* no tiene acceso a la instancia *Tienda*. Una solución consiste en transmitirla hacia abajo (como parámetro) desde *TPDV* (que posee la visibilidad de atributo ante ella) y asignarla a un atributo de *PagoconTarjeta*, para que también tenga visibilidad de atributo frente a ella. Lo anterior es aceptable pero poco práctico; otra opción la constituye el patrón Singleton.

A veces conviene soportar la visibilidad global o un solo punto de acceso a una instancia individual de una clase y no a alguna otra forma de visibilidad. Y resulta que esto ocurre en el caso de la instancia *Tienda*.

Singleton

Contexto/Problema

Se permite exactamente una instancia de una clase: se trata de un "singleton" (solitario). Los objetos necesitan un solo punto de acceso.

Solución

Definir un método de clase o una función que no sea miembro (en C++) y que devuelva el "singleton".

Por ejemplo, como se aprecia en la figura 35.11, suponga que el patrón Singleton se necesita en una *Tienda*. Una solución consiste en definir un atributo de clase *instancia* en la clase *Tienda* que se refiera a una instancia *Tienda* y en definir un método de clase *Instancia()* que retorne el atributo de la clase.¹

Nótese que, en el lenguaje UML los miembros de una clase se indican con el símbolo "\$".

Como se muestra en la figura 35.12, con esta interfaz *PagoconTarjeta* pide a la **clase** *Tienda* visibilidad ante su única instancia y luego ante la instancia de *ServiciodeAutorizaciondeCredito* basándose en una *TarjetadeCredito*.

Puesto que la visibilidad ante las clases tiene un ámbito (relativamente) global, todo objeto puede llamar directamente el método clase que devuelva el elemento considerado Singleton.

¹ Datos y métodos estáticos en el lenguaje Java.

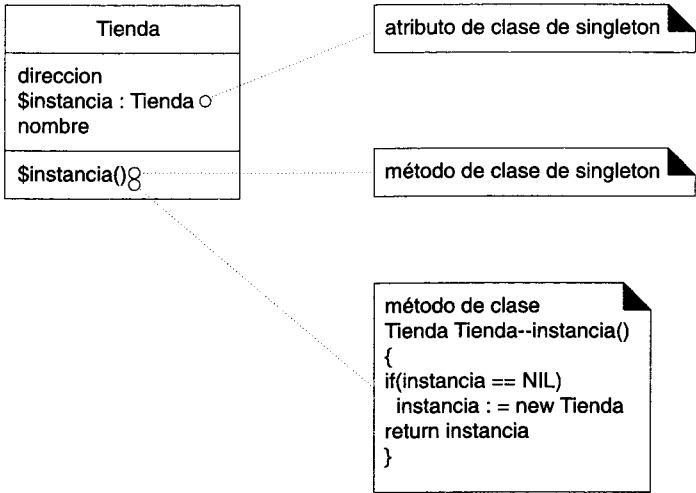


Figura 35.11 Soporte del patrón Singleton en la clase Tienda.

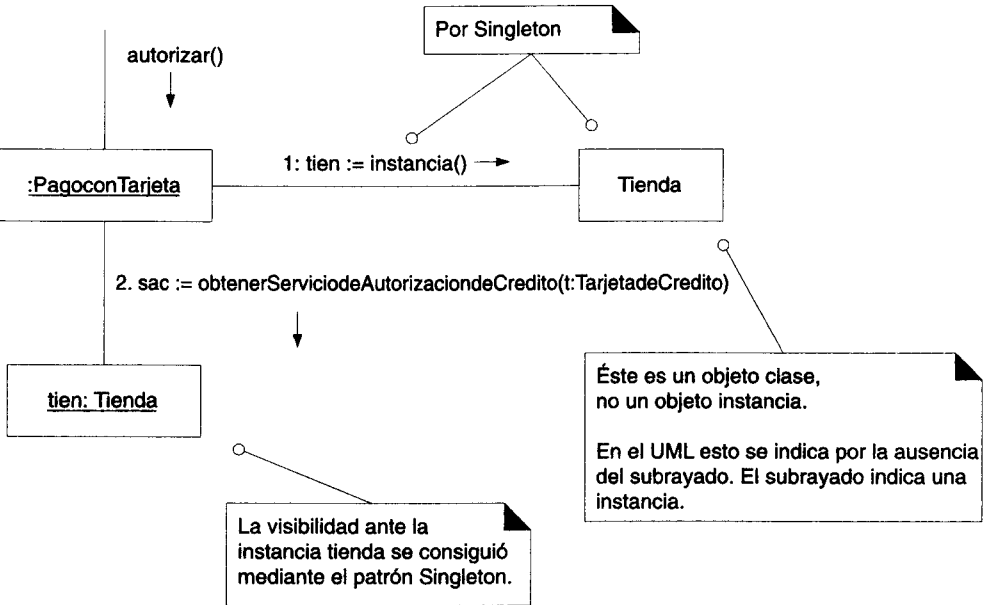


Figura 35.12 Uso del patrón Singleton.

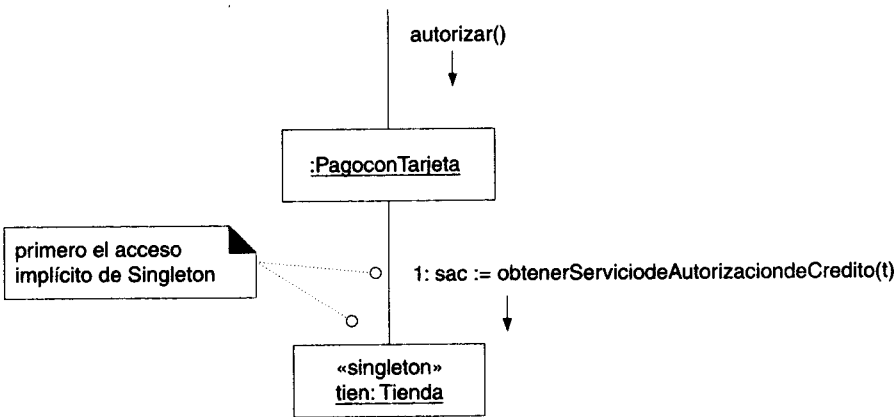
35.4.1 Código muestra

En un lenguaje de programación orientado a objetos, para emplear el patrón Singleton hay que enviar un mensaje a la clase y obtener visibilidad ante la instancia, la cual podrá entonces enviar mensajes.

C++	Tienda::instancia()->obtenerSAC(una-Tarjeta);
Java	Tienda.instancia().obtenerSAC(una-Tarjeta);
Smalltalk	Tienda instancia obtenerSAC: una-Tarjeta.

35.4.2 Abreviatura del patrón Singleton en el UML

Una notación de UML que implica —pero que no necesariamente muestra— el acceso de singleton consiste en estereotipar la instancia con «singleton».



35.4.3 Implementación

- Si se requiere seguridad continua, el método Singleton necesita alguna clase de protección.
- Puntos alternos de acceso:
 - un método de clase en una clase “utilería”
 - (C++) una función que no es miembro

35.5 Agente Remoto y Agente (Pandilla de los Cuatro)

En la sección anterior aludimos a la necesidad de acceder a una instancia de *ServiciodeAutorizaciondeCredito* sin explicar por qué. Recordemos que el sistema debe comunicarse con un servicio externo. En tal situación, el patrón Agente Remoto de la Pandilla de los Cuatro sugiere crear una clase de software local que represente el servicio externo y asignarle la responsabilidad de contactar el servicio real. Por tanto, podríamos usar un objeto local *ServiciodeAutorizaciondeCredito* para hablar con el mundo externo.

Agente Remoto

Contexto/Problema

El sistema debe comunicarse con un componente situado en el espacio de otra dirección. ¿Quién deberá asumir la responsabilidad?

Solución

Crear una clase de software local que represente al componente externo y asignarle la responsabilidad de contactar al componente real.

El Agente Remoto es un caso especial del patrón general Agente (Proxy) de la Pandilla de los Cuatro y sugiere utilizar un sustituto del componente (objeto, servidor, controlador de dispositivo, DLL, etc.) en algunos contextos.

Agente

Contexto/Problema

No se desea o no es posible acceder directamente a un componente. ¿Qué hacer?

Solución

Definir una clase sustituta de software que represente al componente y asignarle la responsabilidad de comunicarse con el componente real.

En nuestro sistema hemos de determinar cuál servicio de autorización emplear y luego, con base en el patrón Agente Remoto, pedirle que se comunique con el servicio real.

Como se aprecia en la figura 35.13, puesto que una *Tienda* es un Experto natural en el conocimiento de sus servicios, pago le pedirá uno de ellos. La clave del servicio corresponde al tipo de compañía de crédito; por eso se transmite *TarjetadeCredito* y se consulta su tipo (*TarjetadeCredito* es un patrón Experto en el conocimiento de su tipo). Y entonces podrá accederse al servicio apropiado.

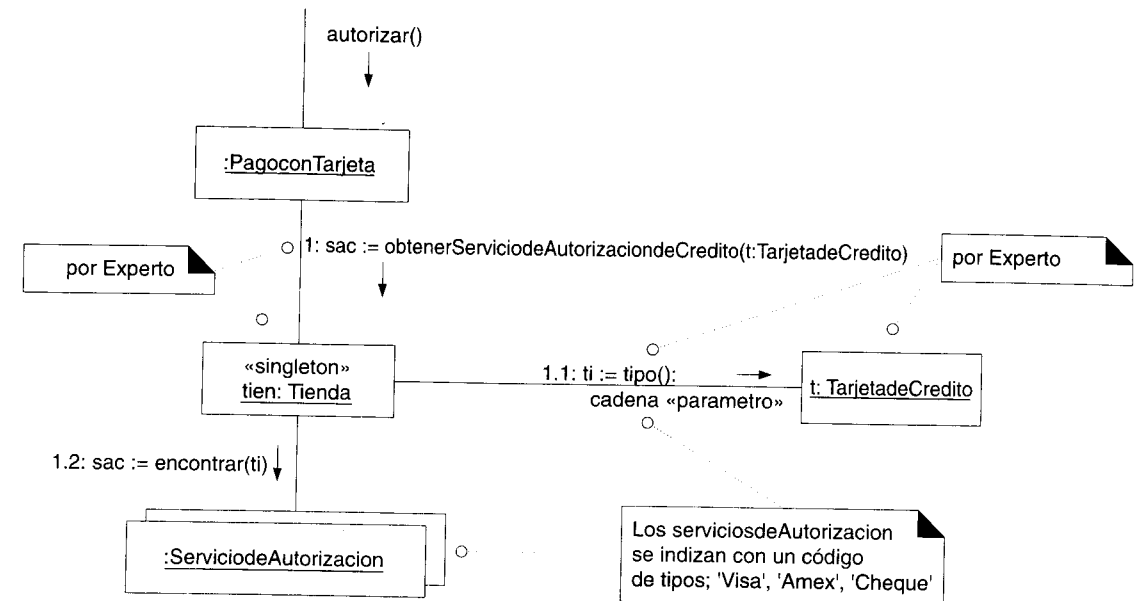


Figura 35.13 Localización de un servicio.

Una vez localizado el Agente Remoto adecuado, se le confiere la responsabilidad de terminar la autorización, según se indica en la figura 35.14.

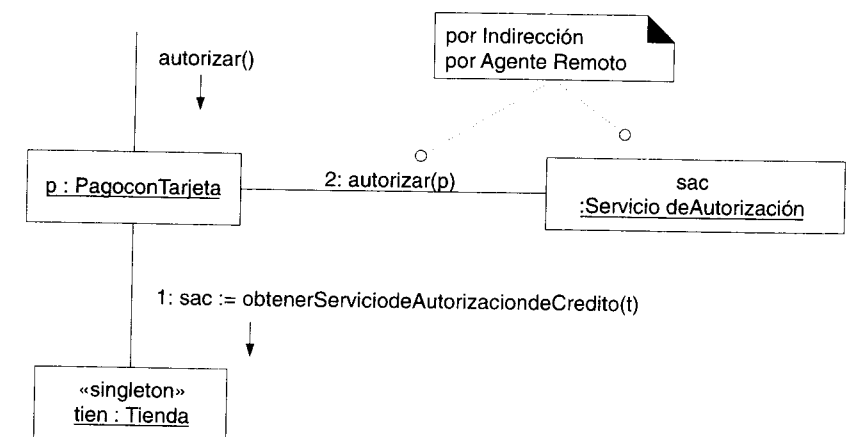


Figura 35.14 Utilización del patrón Agente Remoto.

El Agente Remoto *ServiciodeAutorizaciondeCredito* debe transmitir un mensaje de solicitud y recibir una respuesta, lo cual se hace en la figura 35.15.

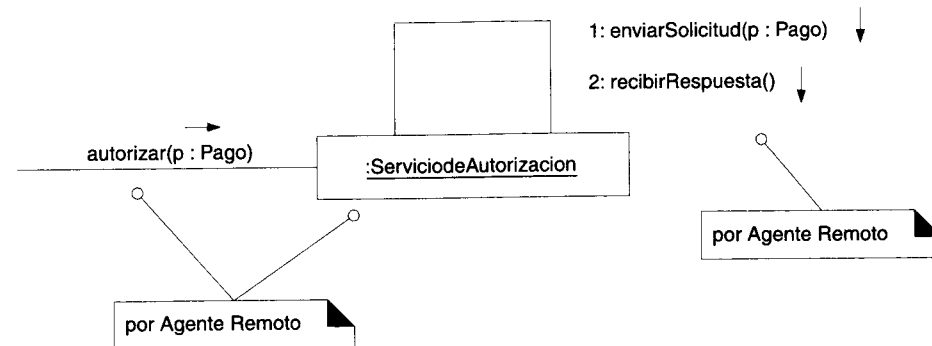


Figura 35.15 Acciones del patrón Agente Remoto.

35.5.1 Problemas

- El lector quizá se haya percatado de que este diseño viola el principio No Hables con Extraños, cuando *PagoconTarjeta* alcanza visibilidad ante el *ServiciodeAutorizaciondeCredito* y le envía el mensaje *Autorizar*. Cuando un objeto es un registro de otra clase de objetos (como lo es *Tienda de ServiciodeAutorizaciondeCredito*), frecuentemente se viola este principio. Una alternativa consiste en darle a la *Tienda* una interfaz *Autorizar(unPago)* para soportar No Hables con Extraños.
- Agente Remoto no es más que uno de tantos agentes. Favor de consultar los detalles en [GHJV95] y en [BMRSS96].
- El Agente Remoto es un simple método de manejar la comunicación. Otras formas basadas en patrones se explican en [BMRSS96].

35.6 El patrón Fachada y el Agente Dispositivo (Pandilla de los Cuatro)

35.6.1 Integración

Nuestro sistema debe utilizar un módem para marcar y conectarse con el servicio externo. Supongamos que el sistema operativo en cuestión ofrece una interfaz basada en funciones para emplear un módem (por ejemplo, *OS_DIAL(...)*). Estas funciones no orientadas a objetos pueden integrarse o quedar “envueltas” dentro de una clase que las agrupe, proporcionando a las funciones una interfaz con el método. En términos generales, a esto se le llama **integración**; puede aplicarse para crear una interfaz orientada a objetos con cualquier otra que no se apegue a este paradigma.

Por ejemplo, podemos definir una clase *Modem* que integre las llamadas.

35.6.2 Fachada

Se le da el nombre de Fachada —uno de los patrones de la Pandilla de los Cuatro— a la clase definida que ofrece una interfaz común con un conjunto heterogéneo de interfaces, entre ellas la clase *Modem*. Las interfaces heterogéneas pueden ser un conjunto de funciones, un esquema, un grupo de otras clases o un subsistema (local o remoto).

Fachada
Contexto/Problema Se requiere una interfaz común unificada con un conjunto heterogéneo de interfaces, como la de un subsistema. ¿Qué debe hacerse?
Solución Definir una sola clase que unifique las interfaces y asignarle la responsabilidad de colaborar con el subsistema.

35.6.3 Agente Dispositivo

Además de ser una Fachada, un *Modem* es además una clase de Agente del módem físico real: un Agente Dispositivo.

Agente Dispositivo
Contexto/Problema Se requiere interactuar con un dispositivo electromecánico. ¿Qué hacer?
Solución Definir una clase que represente al dispositivo y asignarle la responsabilidad de interactuar con él.

35.6.4 Indirección

Observe que Fachada, Agente Remoto y Agente Dispositivo, como tantos otros agentes, constituyen una variante del patrón básico Indirección de GRASP.

35.6.5 Adecuación y serialización

En el diseño orientado a objetos, cuando nos comunicamos con el mundo externo a través de un mecanismo no orientado a objetos, una técnica común consiste en crear objetos que representen los mensajes dirigidos al interior y al exterior para transformarlos luego entre objetos y cadenas. La conversión de un objeto en una representación de cadena recibe el nombre de **serialización**, para la cual algunos lenguajes (Java entre ellos) cuentan con un soporte integrado. Si queremos enviar lo que consideramos un mensaje orientado a objetos con parámetros a través de un medio de comunicación no orientado a objetos (por ejemplo, un "socket"), generalmente debemos transformar el mensaje y los parámetros (mediante la serialización) en un flujo de bytes adecuado para la transmisión y para el servidor receptor. A este proceso se le da el nombre de **adecuación**. Nuestra responsabilidad en la serialización y en la adecuación depende del lenguaje y del mecanismo de comunicación. Si utilizamos Java y su mecanismo de invocación remota del método (RMI), tan sólo habremos de asegurarnos que la serialización produzca una disposición conveniente de cadenas para los parámetros. Cuando no se cuenta con un mecanismo integrado de comunicación distribuida como la invocación remota del método, casi siempre el patrón Agente Remoto se encarga de adecuar y **desadecuar** (transformar en comandos y en objetos las cadenas que retornan).

35.6.6 Utilización del Agente Remoto y del Agente Dispositivo

En conclusión, la clase *Modem* es a la vez un Agente Dispositivo y una Fachada. El Agente Remoto *ServiciodeAutorizacion* crea un objeto que representa al mensaje de salida, lo serializa convirtiéndolo en cadena y luego colabora con el *Modem* para enviarlo afuera. Esto se muestra en la figura 35.16.

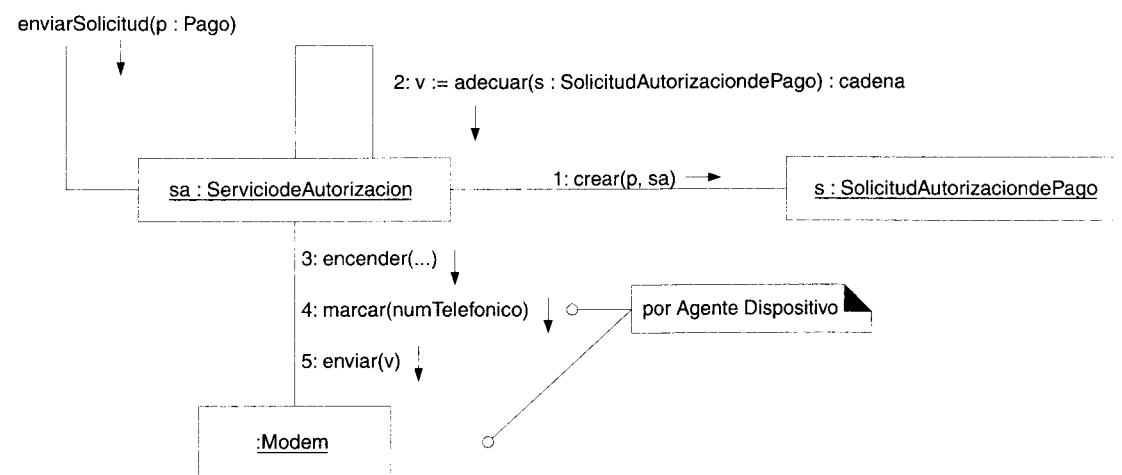


Figura 35.16 Utilización de un Agente Dispositivo.

35.7 El patrón Comando (Pandilla de los Cuatro)

Una vez que el *ServiciodeAutorizacion* envía la solicitud, se dirige también a sí mismo un mensaje *recibirRespuesta* y espera la contestación.¹

Cuando la recibe, este mensaje con formato de cadena se desadecua e introduce en una instancia *RespuestaAprobatoriadePagoconTarjeta* o en una instancia *RespuestaReprobatoriadePagoconTarjeta*, según el código de aprobación que se utilice.

¿Cómo debería efectuarse la ejecución? El patrón Comando de la Pandilla de los Cuatro indica que estos objetos de respuesta representan una clase de solicitud de comando o acción y que han de ejecutarse a sí mismos, basándose para ello en el patrón Polimorfismo.

Comando
Contexto/Problema Un objeto o sistema puede recibir varias peticiones o comandos. Reducir la responsabilidad del receptor en el manejo de los comandos, aumenta la facilidad con que pueden agregarse otros comandos y ofrece las bases para registrar los comandos, para formar colas de espera con ellos y para cancelarlos (deshacerlos).
Solución En cada comando, definir una clase que lo represente y asignarle la responsabilidad de ejecutarse él mismo.

Como se aprecia en la figura 35.17, cada respuesta tiene su propia clase de Comando, con un método *ejecutar* polimórfico.

Cuando se desadecua una respuesta y se la incorpora a la instancia *RespuestaAprobatoriadePagoconTarjeta* o a una instancia *RespuestaReprobatoriadePagoconTarjeta*, se le envía un mensaje *ejecutar*, según vemos en la figura 35.18.

Nótese que esa figura sólo muestra la creación de una superclase abstracta *Respuesta-deAutorizaciondePago*, aun cuando se produjo una instancia de una subclase.

Los casos polimórficos *ejecutar* se incluyen en un nuevo diagrama de colaboración, según se advierte en las figuras 35.19 y 35.20.

¹ También es probable una espera asíncrona en un hilo (thread), pero rebasa el ámbito de nuestra investigación.

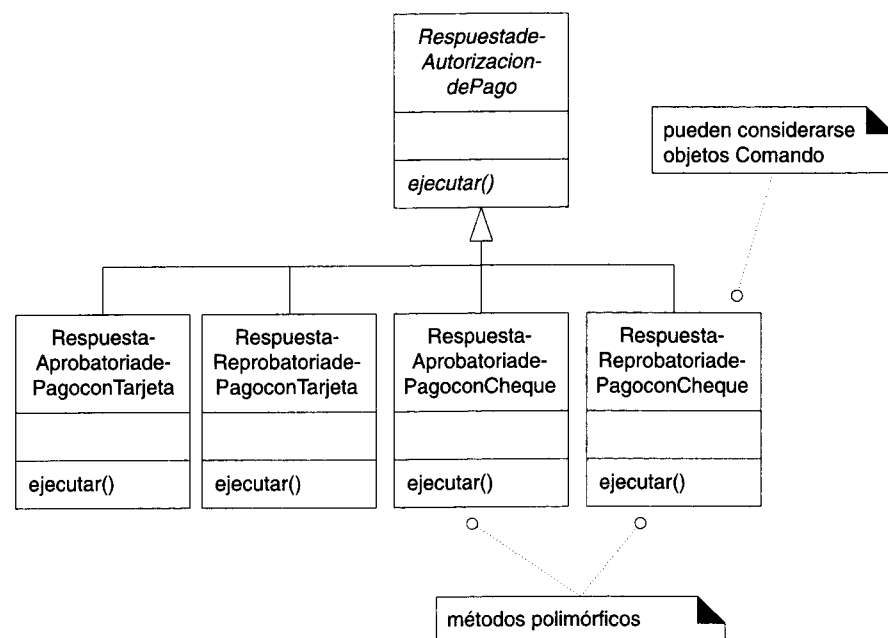


Figura 35.17 Clases de comandos.

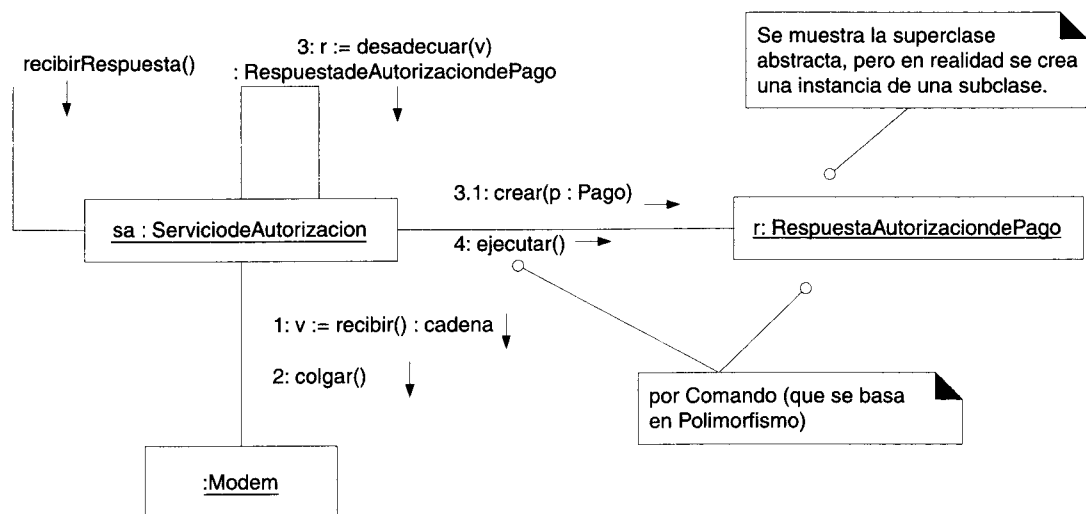


Figura 35.18 Aplicación del patrón Comando.

35.7.1 RespuestaAprobatoriadePagoconTarjeta: ejecutar

En virtud de los patrones Comando y Polimorfismo, la *RespuestaAprobatoriadePagoconTarjeta* recibe un mensaje *ejecutar*. En caso de aprobación, debe registrarse la respuesta en el sistema de cuentas por cobrar. Como se observa en la figura 35.9, esto lo hace por medio de los patrones Singleton, No Hables con Extraños y Agente Remoto.¹

Finalmente, hay que transformar el estado de pago en estado autorizado. (Nótese que a la respuesta se le confirió visibilidad de atributo ante el pago al momento de producir la respuesta.)

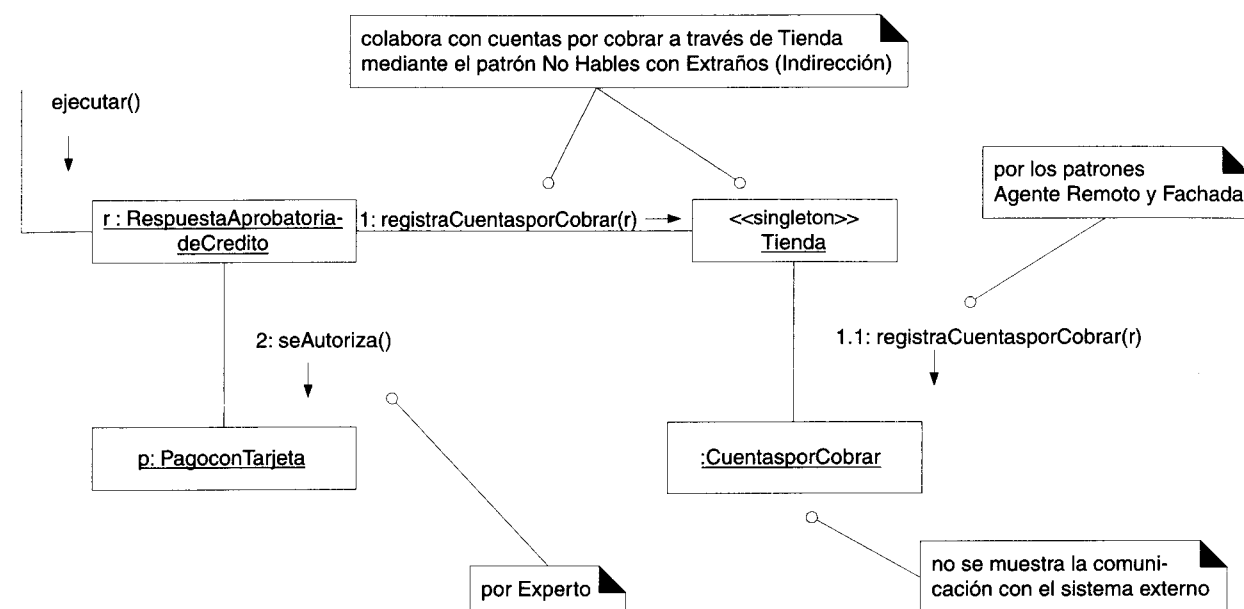


Figura 35.19 Mensaje ejecutar para RespuestaAprobatoriadeCredito.

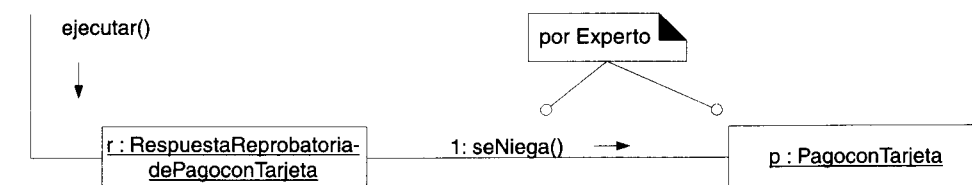


Figura 35.20 Mensaje ejecutar para RespuestaReprobatoriadePagoconTarjeta.

¹ Observe la abstracción posible cuando se comunica un diseño al utilizar los nombres de los patrones.

35.7.2 *RespuestaReprobatoriadePagoconTarjeta: ejecutar*

En virtud de los patrones Comando y Polimorfismo, la instancia *RespuestaReprobatoria-dePagoconTarjeta* recibe un mensaje *ejecutar*. En caso de una respuesta reprobatoria o negativa, ésta ha de cambiar el estado del pago que se rechaza (figura 35.20).

35.8 Conclusión

En este capítulo hemos estudiado varios patrones de la Pandilla de los Cuatro y también hemos hecho aplicaciones de los patrones GRASP.

La principal lección que extraemos de nuestra exposición es la siguiente: durante el diseño orientado a objetos, las responsabilidades pueden asignarse partiendo de la utilización de patrones. Éstos ofrecen un conjunto explicable de técnicas especializadas con las cuales podemos construir sistemas bien diseñados que se orienten a objetos.

PARTE VIII TEMAS ESPECIALES

OTRA NOTACIÓN DE UML

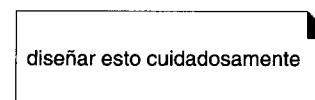
36.1 Introducción

En este capítulo ofrecemos una breve reseña de una parte de la notación del lenguaje UML que todavía no hemos explicado. Recomendamos leer los comentarios del interior de los diagramas para obtener mejor detalle.

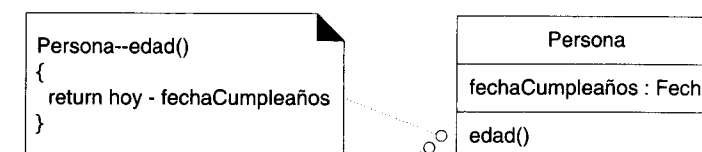
36.2 Notación general

36.2.1 Notas y restricciones

Una nota es un comentario del diagrama. No ejerce influencia semántica sobre los elementos.



Una restricción se anexa a un elemento. Ejerce influencia semántica sobre él.



segundo formato de las restricciones

{hoy – fechaCumpleaños}

Figura 36.1 Notas y restricciones.

36.2.2 Dependencia

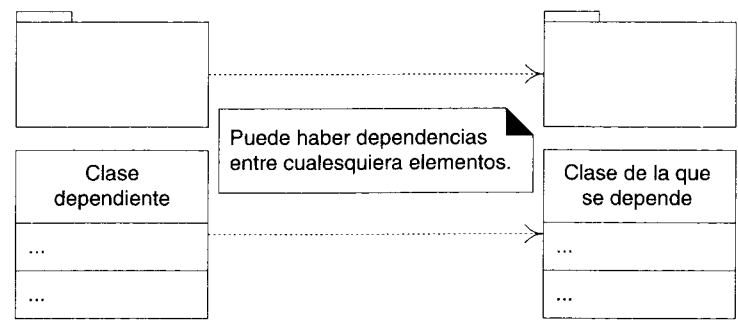


Figura 36.2 Dependencias.

36.2.3 Estereotipos y especificación de las propiedades

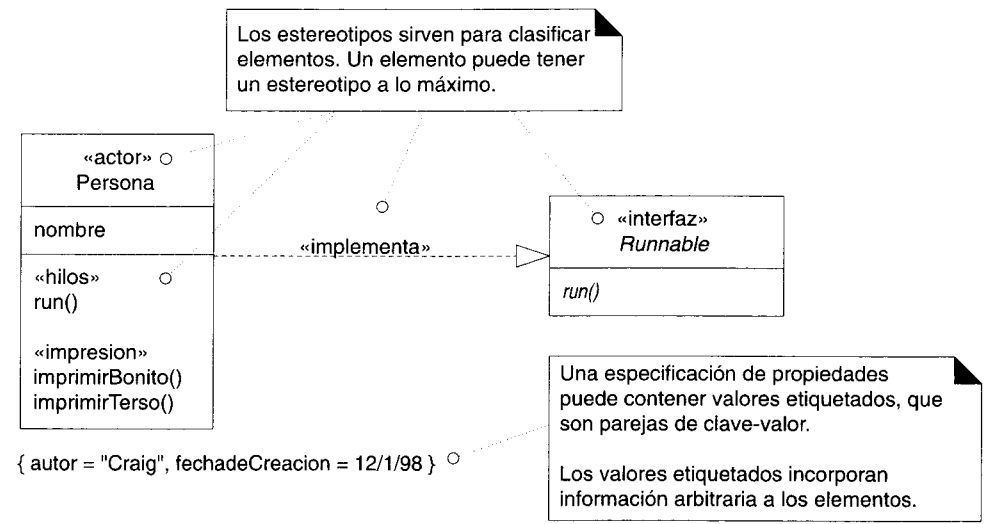


Figura 36.3 Estereotipos y propiedades.

36.3 Interfaces

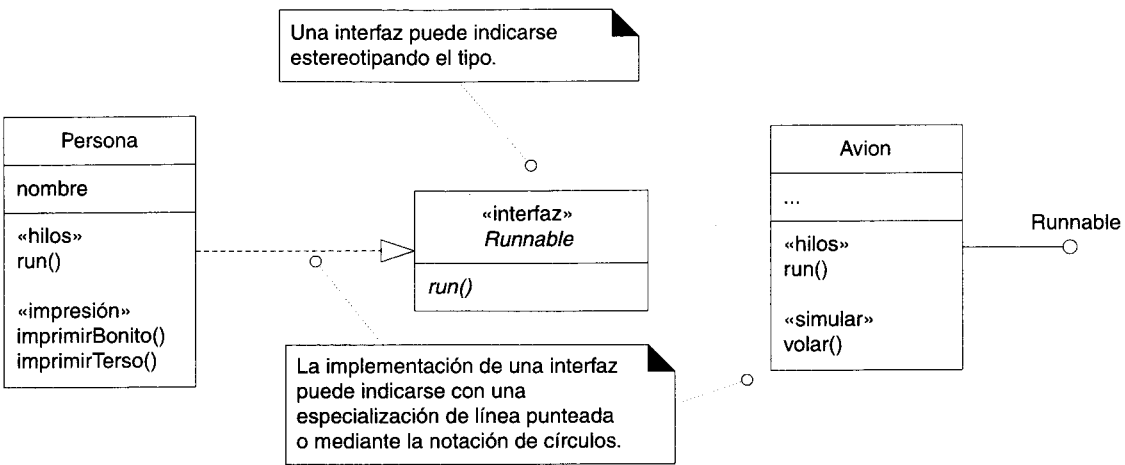


Figura 36.4 Interfaces.

36.4 Diagramas de implementación

36.4.1 Diagramas de componentes

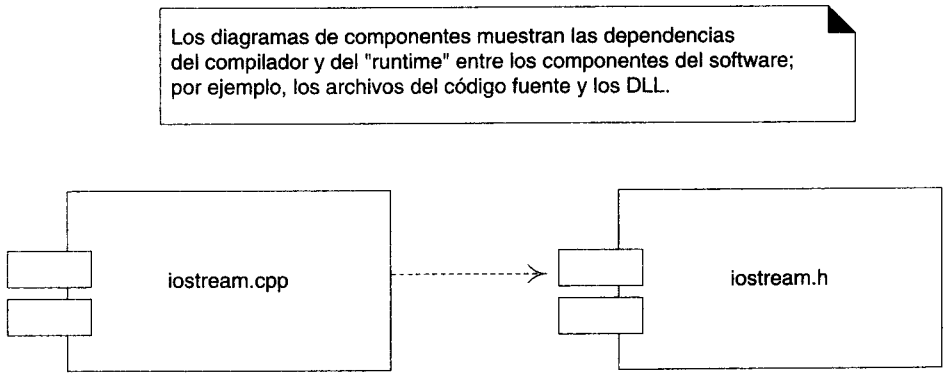


Figura 36.5 Componentes del software.

36.4.2 Diagramas de despliegue

Los diagramas de despliegue muestran a los nodos procesadores la distribución de los procesos y de los componentes.

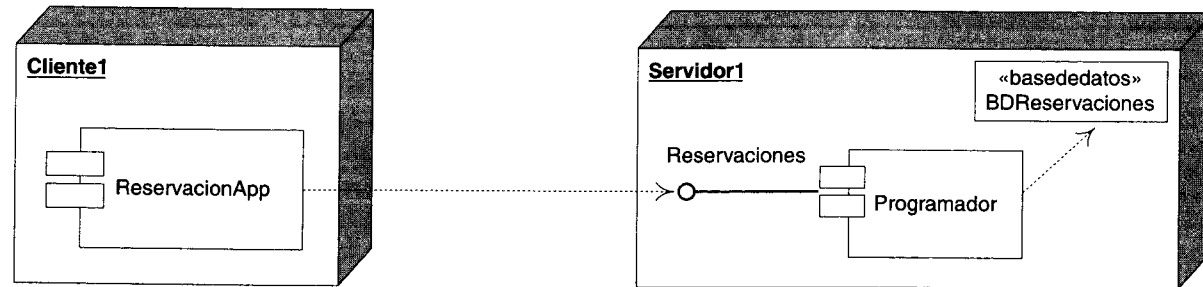


Figura 36.6 Diagrama de despliegue.

36.5 Mensajes asíncronos en los diagramas de colaboración

El lenguaje UML cuenta con una notación que muestra los mensajes asíncronos y los nuevos hilos de control. Por ejemplo, en Java una instancia *Avion* que implementa la interfaz *Ejecutable* (el método *ejecutar*) puede crear un *Hilo* y enviarle un mensaje iniciar (asíncrono). El mensaje ejecutar será entonces devuelto a la instancia *Avion*. El método *ejecutar* es el cuerpo del hilo y *Avion* es un “objeto activo” que ejecuta su propio hilo.

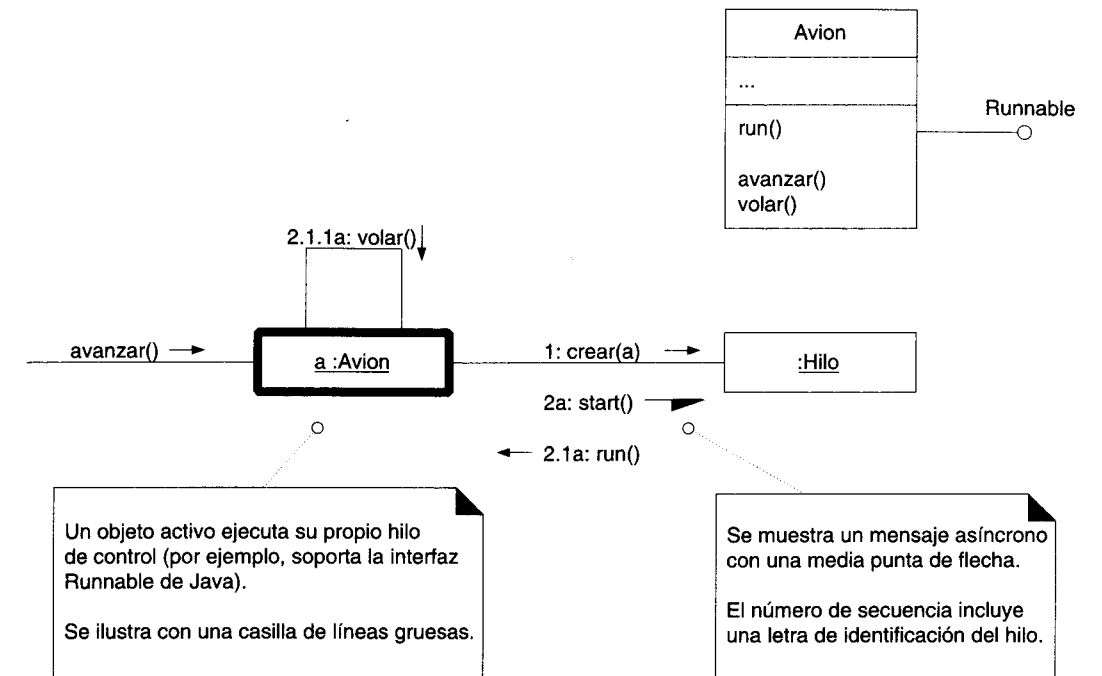


Figura 36.7 Concurrency y los mensajes asíncronos.

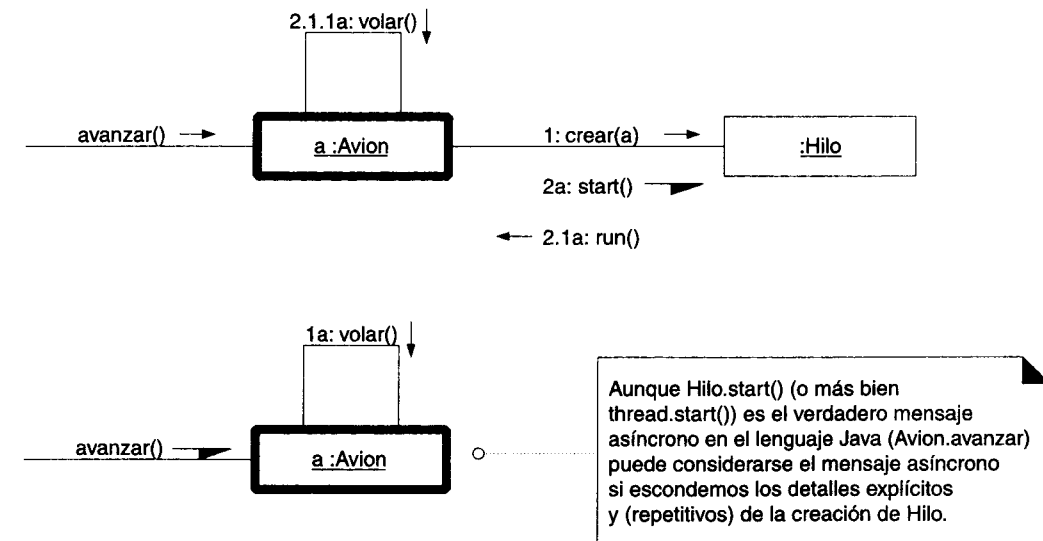


Figura 36.8 Creación implícita de hilos y llamada asíncrona en Java.

36.6 Interfaces de paquetes

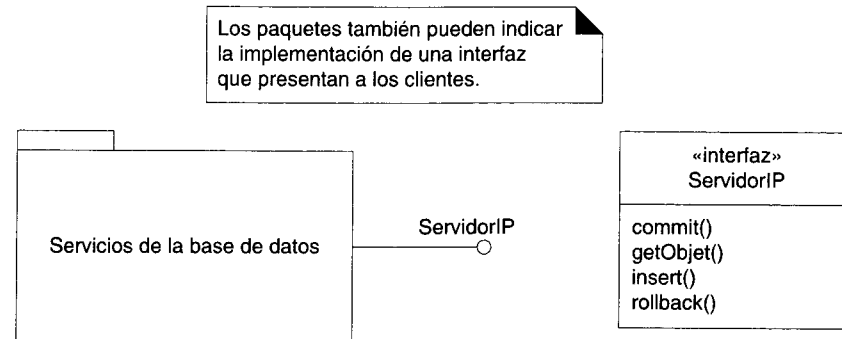


Figura 36.9 Interfaz de un paquete.

PROBLEMAS DEL PROCESO DE DESARROLLO

Objetivos

- Seguir un proceso de desarrollo iterativo, incremental y orientado a los casos de uso.
- Organizar el trabajo a partir de los ciclos de desarrollo.
- Programar un ciclo de desarrollo dentro de un plazo perentorio (o time-box).
- Organizar el desarrollo en equipos paralelos a lo largo de las líneas arquitectónicas.
- Expresar requerimientos técnicos no evidentes como casos de desarrollo de sistemas y calendarizarlos.

37.1 Introducción

En este capítulo estudiaremos algunos problemas fundamentales relacionados con el desarrollo de los sistemas. Vamos a tratar más a fondo algunos temas ya expuestos, repetiremos algunos puntos y luego profundizaremos ideas ya explicadas en capítulos anteriores. Los procesos del desarrollo de software son un tema muy extenso; no sólo abarcan los pasos esenciales de un proyecto, sino también su dirección y muchísimas cuestiones afines. En este capítulo nos limitaremos a algunos de los puntos más importantes.

37.2 ¿Por qué molestarnos?

Con frecuencia oímos verdaderas historias de terror acerca de los fracasos de proyectos de software que han costado millones de dólares: sistemas fiscales, sistemas de defensa y otros. La conclusión salta a la vista: el desarrollo de software es un negocio riesgoso. Un estudio reveló que 31% de los proyectos no se concluyen, que 53% de ellos rebasa casi en 200% el costo estimado, que en 1995 las compañías estadounidenses y las dependencias gubernamentales invirtieron 81 millones de dólares en proyectos de software que fueron cancelados [Standish94]. Simplemente no están obteniéndose los resultados deseados.

Se requiere tiempo y, por tanto, dinero para seguir un proceso de desarrollo y realizar el análisis y el diseño orientados a objetos. ¿Por qué molestarnos? ¿Por qué no iniciar de inmediato la programación? He aquí algunas razones por las cuales debemos tomarnos la molestia de hacer esas dos cosas:

1. Como indican las estadísticas anteriores, los proyectos de software son una actividad riesgosa; de ahí que el motivo primordial sea aminorar el riesgo, esto es, aumentar la probabilidad de crear un sistema eficiente. Y una estrategia para conseguirlo consiste en examinar rigurosamente los requerimientos y elaborar un diseño formal.
2. Conviene crear un proceso que puedan reproducir los individuos, y sobre todo los equipos. No es sustentable el desarrollo que se funda en heroicos esfuerzos individuales.
3. Es más barato y fácil efectuar cambios durante las actividades de análisis y diseño que en la fase de construcción: el software es "más duro" de lo que pensamos.
4. Podemos atenuar y manejar la abrumadora complejidad con sólo modelar los sistemas y hacer abstracciones para detectar los detalles esenciales; así se evita una excesiva complejidad.
5. Conviene crear sistemas robustos, susceptibles de mantenimiento y capaces de soportar una mayor reutilización del software. Por lo regular estas metas no se cumplen sin un riguroso examen y diseño efectuados antes de la programación.

A todo lo anterior le da soporte una aplicación metódica del análisis y del diseño. Por desgracia, muchas compañías no le dan suficiente importancia y son renuentes a invertir recursos en este tipo de actividades, optando generalmente por iniciar cuanto antes el proceso de codificación.

37.3 Directrices de un proceso eficiente

Los siguientes principios forman parte de un proceso recomendable de desarrollo [Booch96]:

- desarrollo iterativo e incremental
- desarrollo orientado a los casos de uso
- desarrollo que comienza por definir la arquitectura ("centrado en la arquitectura")

37.4 Desarrollo iterativo e incremental

37.4.1 Desventajas de un ciclo de vida en cascada

La característica distintiva de un ciclo de desarrollo en cascada es que contiene un *solo* paso de análisis, diseño y construcción; en un único paso se realiza *todo* el análisis, luego *todo* el diseño, luego la codificación y finalmente todas las pruebas. He aquí algunas de sus deficiencias:

- carga frontal de gran complejidad: excesiva complejidad
- retraso de la retroalimentación
- congelamiento temprano de las especificaciones, mientras cambia el ambiente de negocios

37.4.2 Un ciclo de vida iterativo

En contraste con el ciclo de vida en cascada, el ciclo iterativo se funda en el perfeccionamiento gradual de un sistema a través de *múltiples* ciclos de análisis, diseño y construcción, según se aprecia en la figura 37.1.

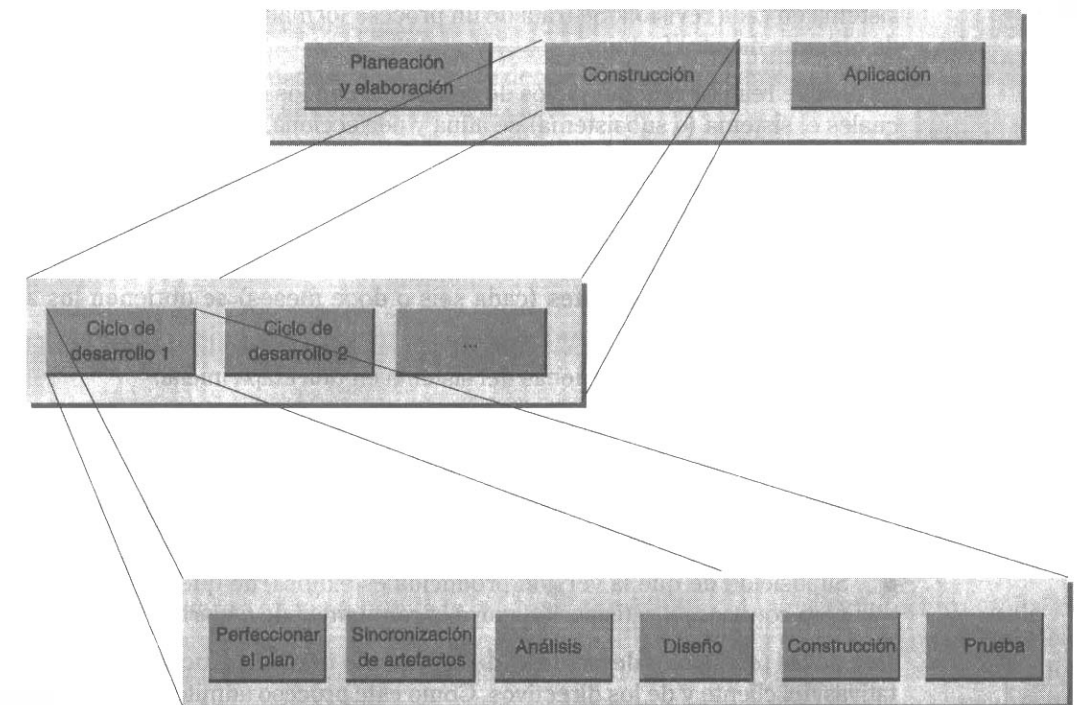


Figura 37.1 Ciclo de vida iterativo.

En cada ciclo se trata un conjunto relativamente pequeño de requerimientos y el sistema crece incrementalmente conforme van concluyéndose los ciclos. Por eso a este proceso se le califica de iterativo e incremental. Aporta los siguientes beneficios:

- La complejidad nunca resulta abrumadora porque en un ciclo sólo se abordan unidades cuya complejidad es manejable. Se evitan así la “parálisis del análisis” y la “parálisis del diseño”.
- Se genera retroalimentación en las fases iniciales, porque la implementación ocurre rápidamente en un pequeño subconjunto del sistema. Esta retroalimentación le suministra información al análisis de ciclos subsecuentes, pudiendo además mejorarlo. Llega a los diseñadores a través de los resultados de su fase de implementación y pruebas o a través de la comunidad que usa una versión ya liberada del sistema.
- El equipo de desarrollo podrá ir reaplicando gradualmente su creciente dominio de las herramientas: no es necesario que desde un principio se sirva de las mejores (y más complejas) características del lenguaje/herramienta ni tampoco que se sirva exclusivamente de técnicas no dominadas para dividir el sistema.
- Los requerimientos son ajustables para que respondan a las necesidades cambiantes de la compañía, a medida que avanza el proyecto.

El desarrollo iterativo no significa hacer una pausa, alcanzar una meta importante y luego volver a detenerse. He aquí una definición más exacta del **desarrollo iterativo**: es un proceso *planeado* que consiste en revisar varias veces un área, mejorando el sistema en cada revisión. Se trata de un proceso formalmente planeado y programado, de ninguna manera fortuito.

Es posible realizar muchos ciclos de desarrollo con los mismos requerimientos, en los cuales el sistema (o subsistema) se afina y perfecciona, y también hacerlo con nuevos requerimientos (que es el caso más común).

El **desarrollo incremental** consiste en agregarle funcionalidad a un sistema en los ciclos productivos; con cada uno va aumentando la funcionalidad. Una creación incremental se compone de varios ciclos de desarrollo iterativo. Con producciones incrementales bastante frecuentes (cada seis o doce meses) se obtienen los siguientes beneficios:

- Integración global y pruebas del sistema en una etapa inicial.
- Retroalimentación temprana al diseñador por parte de la comunidad de usuarios finales, utilizando el sistema generado.
- Retroalimentación al cliente y a la comunidad de usuarios finales respecto a la habilidad y confiabilidad del equipo encargado del desarrollo.
- Suposición de que la versión producida es exitosa, de que desde un principio genera confianza y satisfacción entre la comunidad de usuarios.

Una de las principales desventajas del desarrollo iterativo e incremental son las expectativas del cliente y de los directivos. Como este proceso admite demostraciones en las primeras fases, el cliente y los directivos tenderán a pensar que el sistema “casi está terminado”: el conocido problema de “funciona la interfaz para el usuario = sistema

terminado”. Se requiere de un manejo riguroso de las expectativas y una comunicación frecuente y clara por parte del equipo de desarrollo, si se quiere atenuar esta reacción, desgraciadamente inevitable.

37.5 Desarrollo orientado a los casos de uso

El proceso de desarrollo debería evidenciar el influjo de los casos de uso y organizarse en torno a ellos. Por ejemplo:

- Los requerimientos se organizan y se redactan a partir de los casos de uso.
- En las estimaciones influyen directa e indirectamente los casos: su número, su complejidad, los servicios de soporte que requieren, etcétera.
- Los programas se organizan a partir de ciclos iterativos, los cuales se basan en la realización de los casos de uso.
- A partir de los casos de uso se escogen los requerimientos que deberían cumplirse en un ciclo iterativo particular.
 - Por ejemplo, en la iteración 1 nos ocuparemos del caso A, en la iteración 2 nos ocuparemos de una versión simplificada del caso B y en la iteración 3 nos ocuparemos del resto del caso B e íntegramente de los casos C y D.
- Las actividades de un ciclo de desarrollo procuran ante todo llevar a cabo el caso o casos que se consideran en el ciclo.
- Los conceptos y las clases de software pueden identificarse atendiendo a los casos de uso.
- Se escriben casos de prueba para validar los que se llevan a cabo.

37.6 Énfasis inicial en la arquitectura

Dentro del contexto de este capítulo, con el término **arquitectura** designamos la estructura de alto nivel de los subsistemas y de los componentes, así como de sus interfaces, conexiones e interacciones. La arquitectura se compone de un conjunto de esquemas, subsistemas, clases, asignaciones de responsabilidades y colaboraciones entre objetos que cumplen con las funciones del sistema.

Un proceso eficaz estimula la generación *temprana* de una arquitectura global del sistema: está centrado en la arquitectura. Ello significa que en los primeros ciclos se ponen de relieve la investigación y el desarrollo de esquemas arquitectónicos bien diseñados; al mismo tiempo se tiene una visión global de una arquitectura cohesiva a lo largo del proyecto. El diseño se ha construido en una forma sistemática y organizada.

Una arquitectura bien diseñada reúne las siguientes cualidades ideales:

- tiene subsistemas sustentados en arquitectura de capas
- ofrece bajo acoplamiento entre subsistemas
- es fácil de entender
- es robusta, flexible y se incrementa de modo adecuado
- tiene muchos componentes reutilizables
- se orienta a los casos de uso más importantes y riesgosos
- consta de interfaces bien definidas con el sistema y con los subsistemas

Durante el desarrollo temprano, deberá darse prioridad a las capas y a los subsistemas con mayor riesgo e incertidumbre. Pueden conformar la capa del dominio o alguna de las capas de servicios de alto nivel. No obstante, la importancia especial que se concede a la arquitectura significa que desde un principio se atenderá a la mayoría de las capas y de los subsistemas. Esto incluye sus componentes, sus interfaces e interacciones con otras capas.

37.7 Fases del desarrollo

37.7.1 Versiones de producción

En el macronivel, un proyecto de desarrollo consta de una o más versiones incrementales de producción: una versión que se destina al uso. En el caso del software comercial, ello significa una versión que se vende; en el caso del software de una empresa, significa un uso operacional.

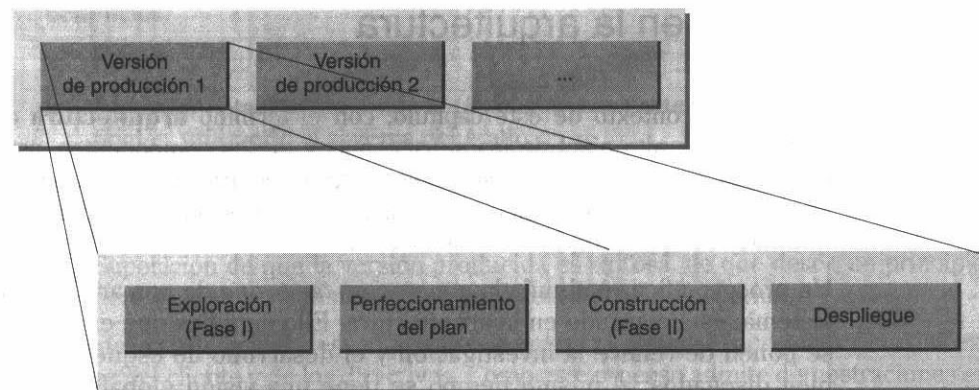


Figura 37.2 Desarrollo incremental del producto.

Una versión incremental de producción consta de las siguientes fases:

1. **Explorar:** se definen los requerimientos y se construye el sistema a través de varios ciclos de desarrollo hasta alcanzar un punto donde puede formularse un plan aceptable.
2. **Perfeccionar el plan:** se modifica el programa del proyecto, sus presupuestos y otros aspectos, basándose para ello en los resultados de la exploración.
3. **Construir:** es la fase principal de la terminación de desarrollo.
4. **Desplegar:** se traslada el sistema al uso de producción.

37.7.2 Principales pasos del desarrollo

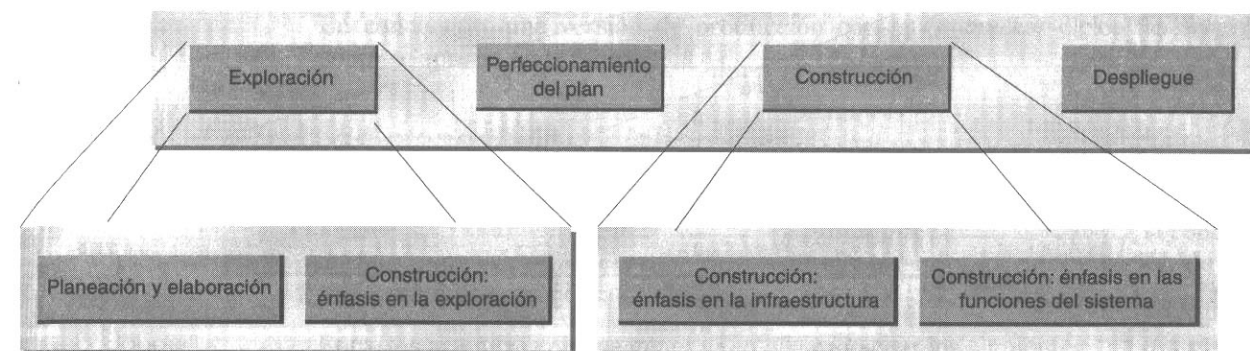


Figura 37.3 Principales pasos del desarrollo.

En una versión de producción, la fase Explorar (también llamada fase I) se propone entender los requerimientos y luego examinar la construcción de un sistema hasta lograr generar una estimación y un programa aceptables.¹ Esta fase está constituida por dos grandes etapas:

1. **Planeación y elaboración:** los requerimientos iniciales de obtención, definición, planeación y formulación del problema.
2. **Construcción: énfasis en la exploración.** Un periodo de uno a tres meses de los ciclos de desarrollo en que se llevan a cabo algunos casos de uso y trabajo de arquitectura de alto nivel.

La importante fase de construcción (llamada también fase II) tiene por objeto desarrollar íntegramente el sistema. Los ciclos de desarrollo de que consta pueden agruparse así:

1. **Construcción: énfasis en la infraestructura.** Son los ciclos iniciales de desarrollo cuyo fin es establecer y perfeccionar la arquitectura técnica global, como

¹ Son mera ficción las estimaciones y los programas de actividades obtenidos sin ciclos exploratorios de desarrollo que midan concretamente las dimensiones y dificultad del problema, así como la rapidez del equipo de desarrollo.

esquemas (frameworks), capas y subsistemas. Esta fase *sí* incluye realizar los casos del dominio (*Comprar Productos*, entre ellos), pero se procura ante todo sentar las bases arquitectónicas que soporten su realización.¹

2. **Construcción: énfasis en las funciones del sistema.** Son los ciclos restantes que subrayan el cumplimiento de las funciones de aplicación, durante los cuales queda por hacer poco trabajo de infraestructura.

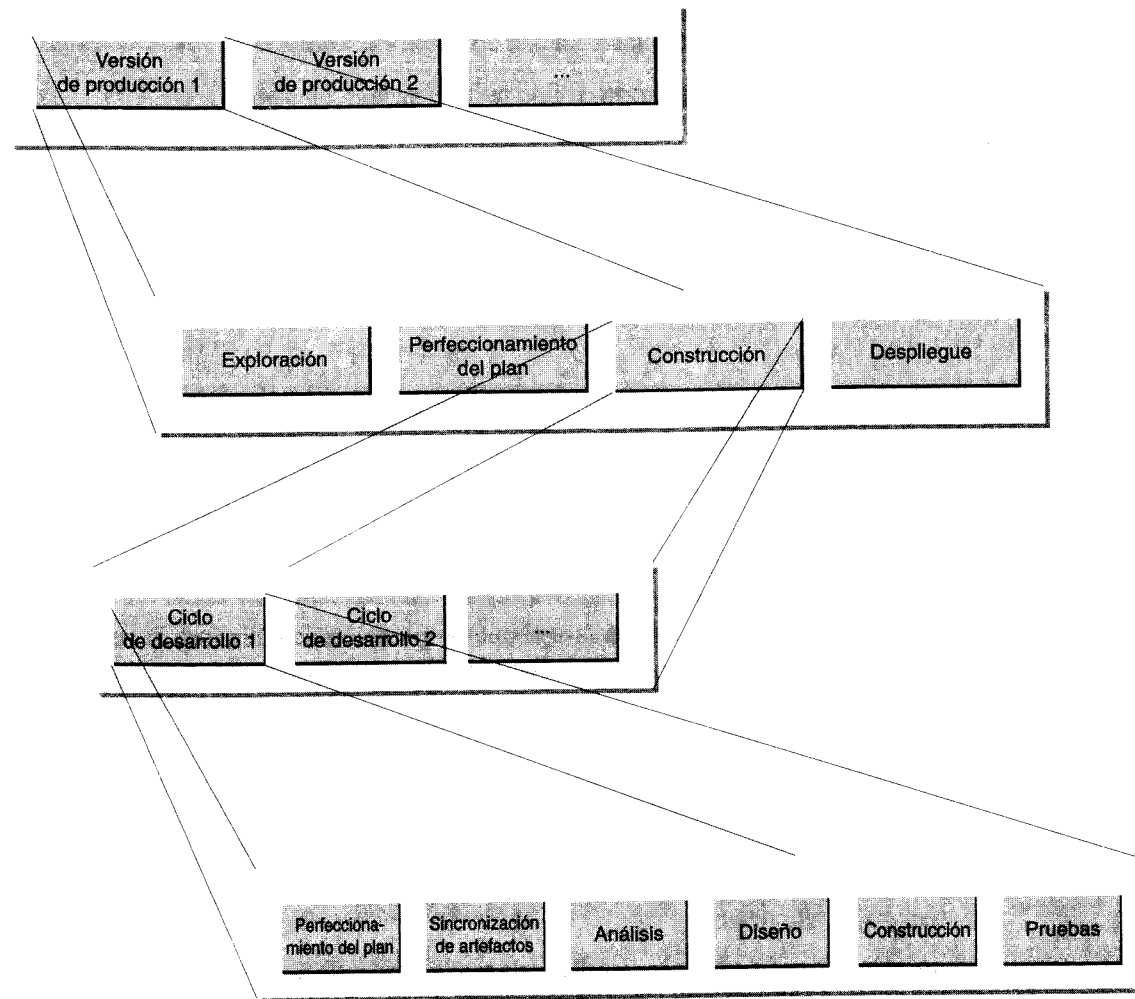


Figura 37.4 Ciclos de desarrollo iterativo.

¹ A veces se subestima mucho el tiempo necesario para crear la infraestructura y concluir esta fase.

37.7.3 Comparación entre la versión de producción y los ciclos de desarrollo

Un error muy frecuente en el desarrollo iterativo consiste en confundir una versión de producción con un ciclo de desarrollo.

Por ejemplo, desde el punto de vista del desarrollo iterativo no conviene tener un solo ciclo de desarrollo semestral y luego generar un sistema de producción al final de ese periodo.

Por el contrario, si hay que entregar un sistema de producción cada seis meses, ese lapso debería dividirse en —por ejemplo— seis ciclos mensuales: seis ciclos para analizar, diseñar, implementar y efectuar pruebas.

En conclusión, una versión de producción consta de *muchos* ciclos de desarrollo, como se aprecia en la figura 37.4.

37.7.4 La fase de planeación y elaboración

La fase de planeación y elaboración de un proyecto (figura 37.5) abarca la concepción inicial, la investigación de alternativas, la planeación y la especificación de requerimientos entre otras cosas.

Entre los artefactos que se generan en ella cabe citar los siguientes:

- **Plan:** programa del proyecto, recursos, presupuesto y otros elementos.
- **Informe preliminar de investigación:** motivos, opciones, necesidades de la empresa, etcétera.
- **Especificación de requerimientos:** declaración de los requerimientos.
- **Glosario:** un diccionario de términos (nombres de conceptos y otros vocablos) e información relacionada con ellos; por ejemplo, reglas y restricciones.
- **Prototipo:** un sistema de prototipos cuya finalidad es facilitar la comprensión del problema, de los problemas de alto riesgo y de los requerimientos.
- **Casos de uso:** descripciones textuales de los procesos del dominio.
- **Diagramas de los casos de uso:** descripción visual de todos los casos y de sus relaciones.
- **Modelo conceptual preliminar:** modelo inicial que ayuda a entender el vocabulario del dominio, sobre todo en su relación con los casos de uso y con la especificación de requerimientos.

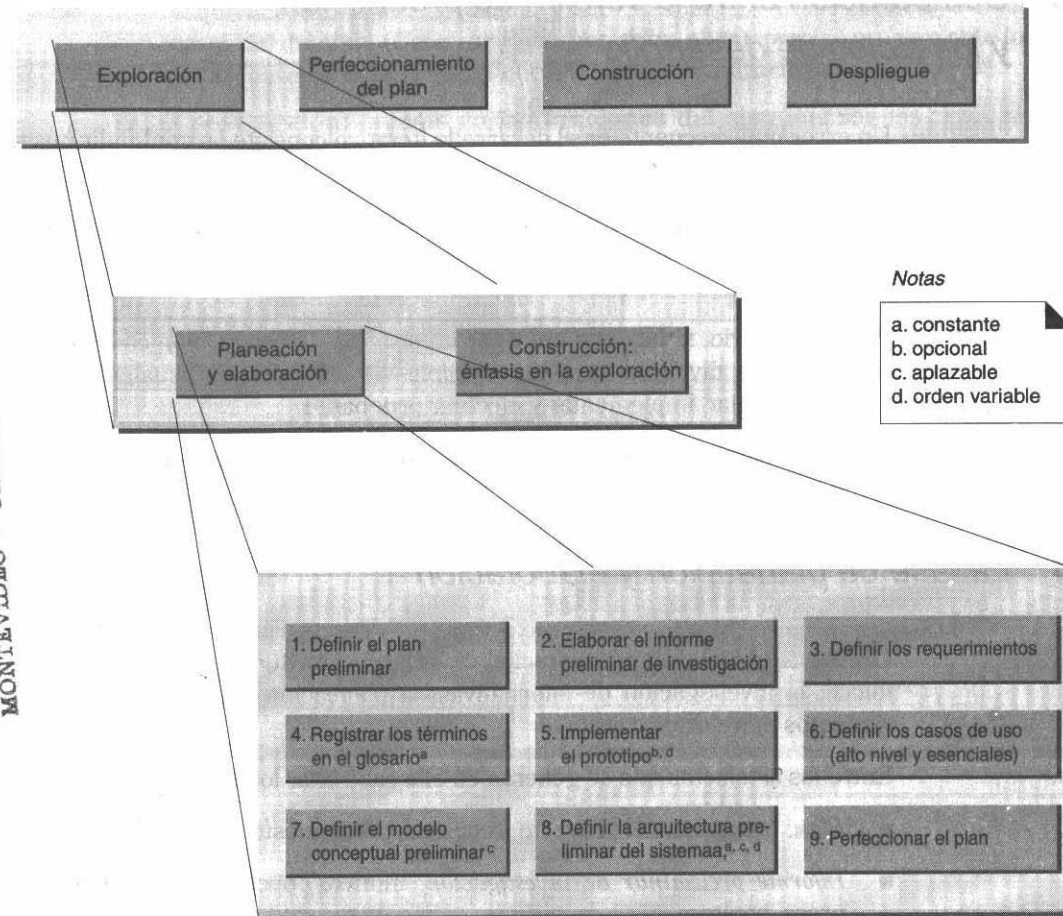


Figura 37.5 Fase de planeación y elaboración del desarrollo.

37.7.5 Fase de construcción: énfasis en la exploración

Esta fase, que aparece en la figura 37.6, incluye los ciclos de desarrollo de uno a tres meses. Se examinan los casos de uso de alto nivel que influyen profundamente en la arquitectura. Con ello se busca explorar lo suficiente como para recabar información, a partir de la cual idear un programa y una estimación confiables para el proyecto.

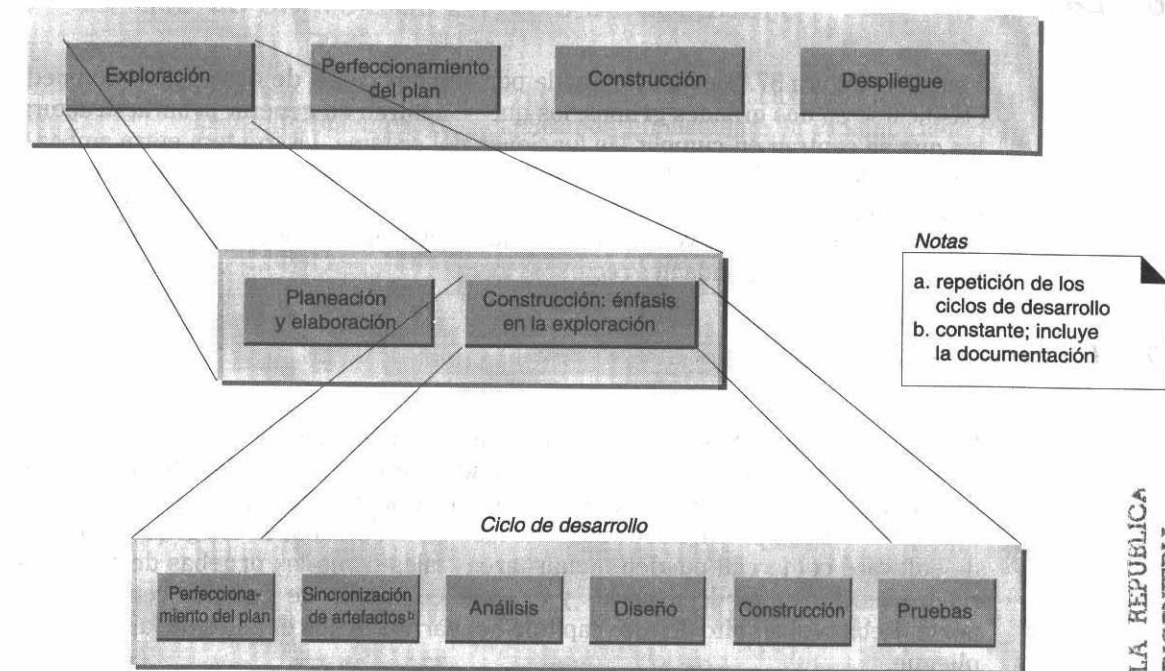


Figura 37.6 Actividades del ciclo de desarrollo.

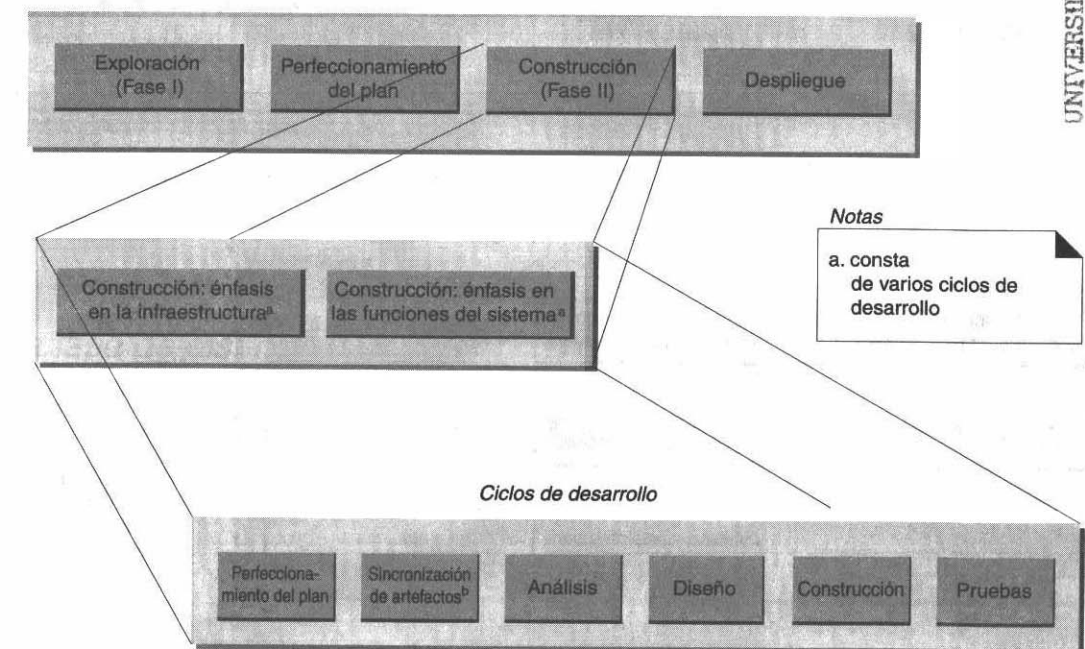


Figura 37.7 La fase de construcción consta de varios ciclos de desarrollo.

37.7.6 La fase primaria de construcción

Esta fase (figura 37.7) está constituida por muchos ciclos de desarrollo que pueden clasificarse en dos grandes grupos: los que se centran en generar la infraestructura y los que se centran en cumplir las funciones del sistema. La frontera entre ambos no está bien definida: durante los ciclos de desarrollo orientados a la infraestructura, uno o más equipos trabajarán en las funciones del sistema y a la inversa. Con esta precisión deseamos poner de relieve que, en muchos proyectos, el esfuerzo dedicado a sentar las bases requiere abundantes recursos.

37.7.7 La fase de despliegue

Esta fase (figura 37.8) consiste en introducir el sistema en la etapa de producción. Si se trata de software comercial, ello significa que se venderá a los clientes para que lo utilicen; si se trata de software para la empresa, significa que formará parte del uso operacional. Las actividades de esta fase varían mucho; en los grandes sistemas de software comercial pueden incluir tareas tales como las pruebas de aceptación realizadas por miles de usuarios y el establecimiento de grandes centros de soporte. Rebasa el ámbito de este capítulo ocuparnos de los detalles de la fase de despliegue.

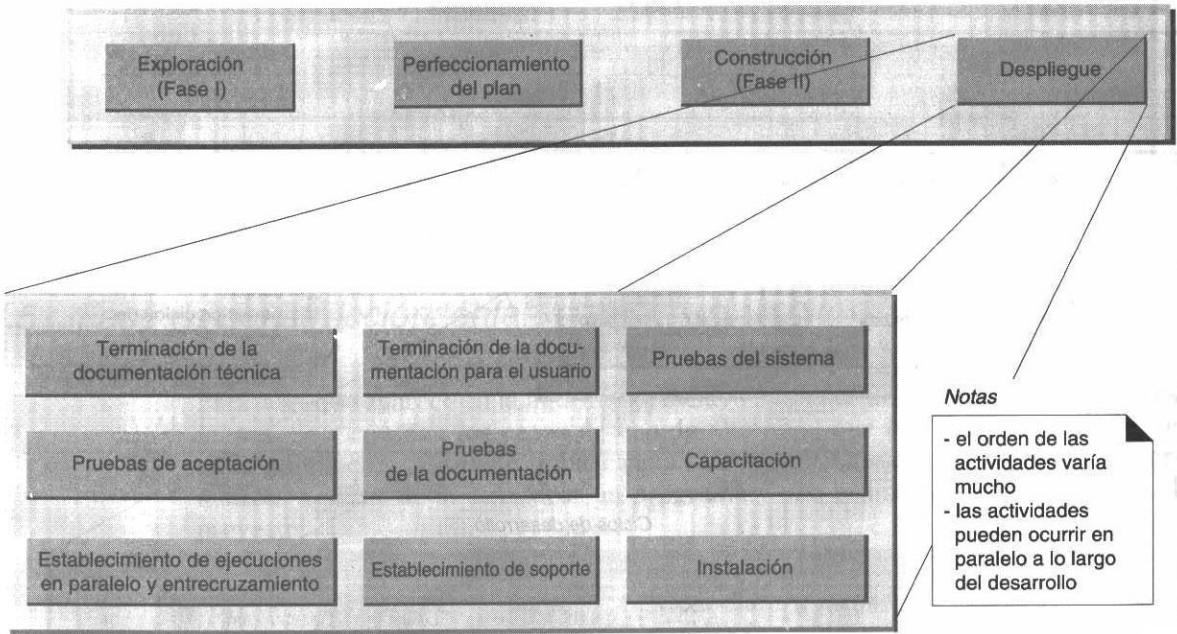


Figura 37.8 La fase de despliegue.

37.7.8 La fase de análisis de un ciclo de desarrollo

Esta fase (figura 37.9) se centra en una investigación del problema y en el análisis de los requerimientos. Ya la explicamos ampliamente en capítulos anteriores.

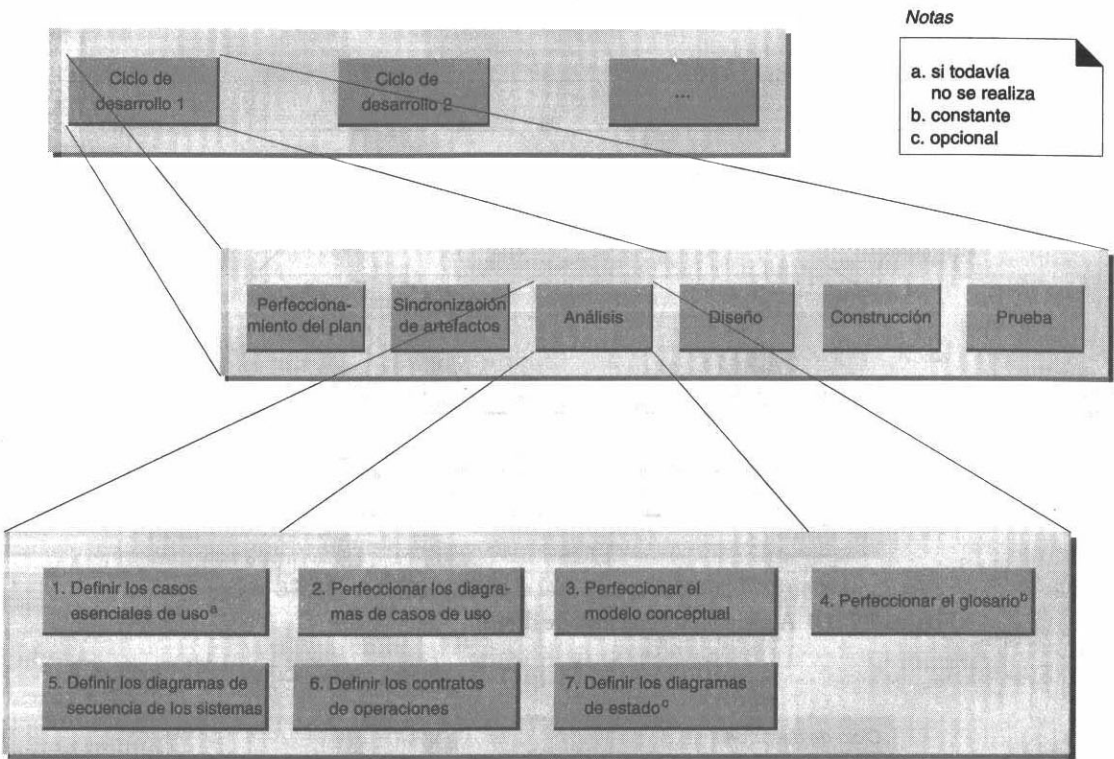


Figura 37.9 Actividades de la fase de análisis.

37.7.9 La fase de diseño de un ciclo de desarrollo

En esta fase se busca ante todo definir una solución lógica (figura 37.10). Ya la estudiamos detenidamente en capítulos anteriores.

37.7.10 La fase de construcción de un ciclo de desarrollo

Esta fase requiere implementar el diseño en software y en hardware (figura 37.11).

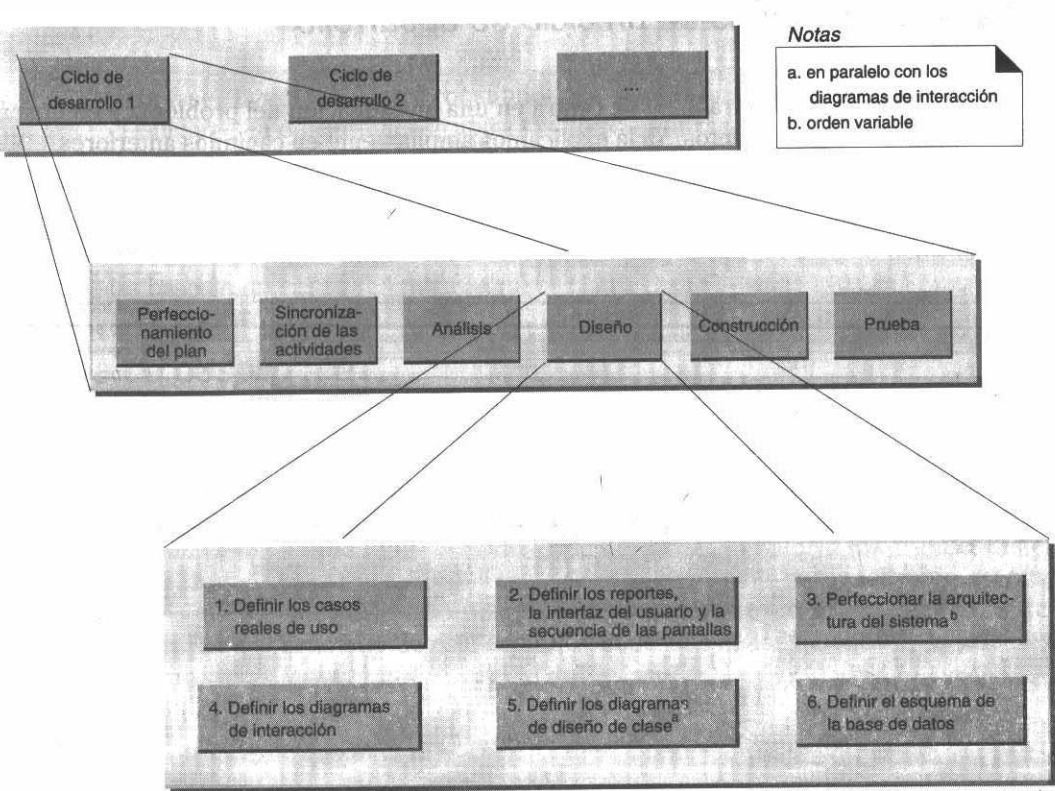


Figura 37.10 Actividades de la fase de diseño.

37.7.11 La fase de pruebas

Aunque esta fase se incluye como paso final dentro de un ciclo de desarrollo, también se recomienda como una actividad constante en la fase de construcción. No todas las pruebas mencionadas en la figura 37.12 son adecuadas en los ciclos de desarrollo. Por ejemplo, las pruebas de integración del sistema entero pueden efectuarse con menor frecuencia en cada uno.

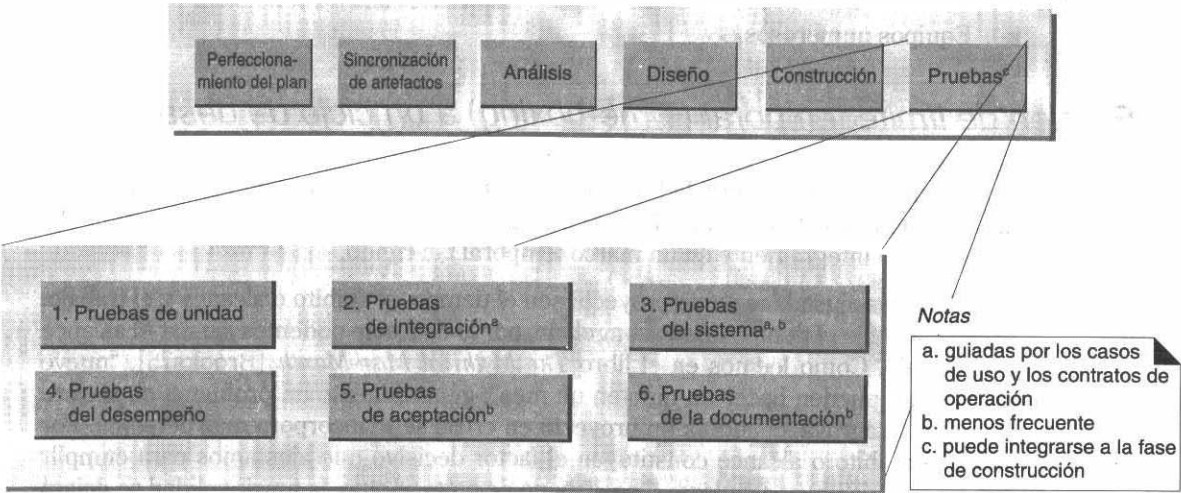


Figura 37.12 Actividades de la fase de realización de pruebas.

37.8 Duración de los ciclos de desarrollo

Con una duración menor de la necesaria será difícil realizar un conjunto significativo de requerimientos y actividades; con una duración mayor se correrá el riesgo de una excesiva complejidad y de insuficiente retroalimentación inicial.

Un ciclo de desarrollo debería durar entre dos semanas y dos meses.

En igualdad de circunstancias, aconsejamos optar por un ciclo de dos a cuatro semanas. A continuación se mencionan algunos factores que pueden hacer que un ciclo se prolongue hasta durar dos meses:

- Ciclos iniciales de desarrollo en que empiezan a desarrollarse la infraestructura y los servicios básicos, y además se lleva a cabo una abundante investigación exploratoria. Advertencia: muchas veces se subestima el tiempo que se tardará en concluir la infraestructura.
- Introducción de tecnología o de métodos nuevos.

- Inaccesibilidad a los expertos en el dominio del tema.
- Importante número de procesos distribuidos o concurrentes o trabajo de comunicación.
- Personal novato (en la tecnología de objetos, en el dominio del problema o en el desarrollo iterativo).
- Equipos de desarrollo en paralelo.
- Equipos de desarrollo físicamente distribuidos que trabajan en paralelo.¹
- Equipos numerosos.

37.8.1 Fijación de límite temporal (Time-boxing) a un ciclo de desarrollo

Una estrategia de gran utilidad en los ciclos de desarrollo consiste en establecer un límite temporal, esto es, un periodo fijo, digamos cuatro semanas. El trabajo debe llevarse a cabo íntegramente en un marco temporal tan rígido.

Tres factores ajustables en un proyecto son el tiempo, el ámbito o alcance y el trabajo. En un plazo fijo el tiempo queda congelado, por lo cual sólo podemos ajustar el alcance y el trabajo. Como leemos en el libro *The Mythical Man-Month* [Brooks75], “nueve mujeres no pueden hacer un niño en un mes”; generalmente un problema no mejora sino que se agrava, cuando a un proyecto en crisis se le incorpora más personal. Por tanto, el ámbito o alcance constituyen el factor decisivo que ajustamos para cumplir con las restricciones del plazo; si se dispone de poco tiempo, la funcionalidad se dejará para un ciclo ulterior de desarrollo.

El equipo de desarrollo habrá de decidir cuáles requerimientos cumplir en el plazo; son ellos los que deben presentar los resultados.

37.9 Aspectos del ciclo de desarrollo

37.9.1 Equipos y ciclos de desarrollo en paralelo

Un proyecto extenso suele dividirse en actividades de desarrollo en paralelo, donde varios equipos trabajan de modo simultáneo (figura 37.13).

Una forma de organizar los equipos en líneas arquitectónicas es hacerlo por capas y subsistemas. Otra forma consiste en basarse en un conjunto de características, que pueden corresponder muy bien a la organización arquitectónica. Por ejemplo:

¹ Tener un equipo en otro piso del mismo edificio produce casi el mismo impacto que si estuviera en un sitio geográficamente distante.

- Equipo de la capa del dominio (o equipo del subsistema del dominio)
- Equipo de interfaz para el usuario
- Equipo de internacionalización
- Equipo de la capa de servicios de alto nivel (equipo de persistencia, equipo de presentación de reportes, etcétera)

Estos equipos pueden trabajar en ciclos de desarrollo en paralelo; todos concluirán un ciclo en la misma fecha.

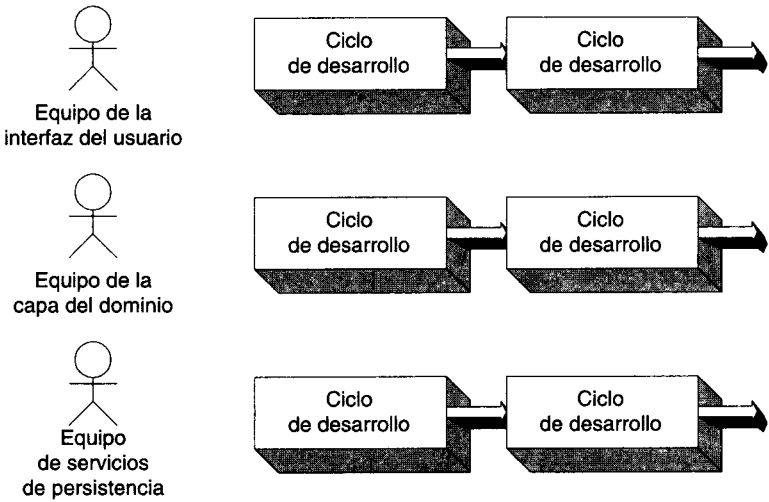


Figura 37.13 Equipos de desarrollo en paralelo.

En capas muy complejas donde el avance siempre es lento al inicio (entre ellas, los servicios de persistencia), se recomienda aumentar la duración del ciclo de desarrollo (figura 37.14). De ese modo, los equipos podrán trabajar en ciclos de duración variable. Todas ellas deberán ser múltiplos del ciclo más corto.

37.9.2 Requerimientos no evidentes y la arquitectura técnica

Algunos requerimientos pueden estar implícitos, o sea que no son explícitamente evidentes en la especificación de requerimientos ni en los casos de uso. Ello sucede sobre todo al diseñar una arquitectura técnica global y servicios de alto nivel, como los de persistencia.

Se requiere mucho trabajo para diseñar e implementar la arquitectura técnica global de un sistema: capas arquitectónicas, despliegue físico, cómputo distribuido, etc. Hay que identificar y programar este trabajo: en la siguiente sección se explica cómo hacerlo.

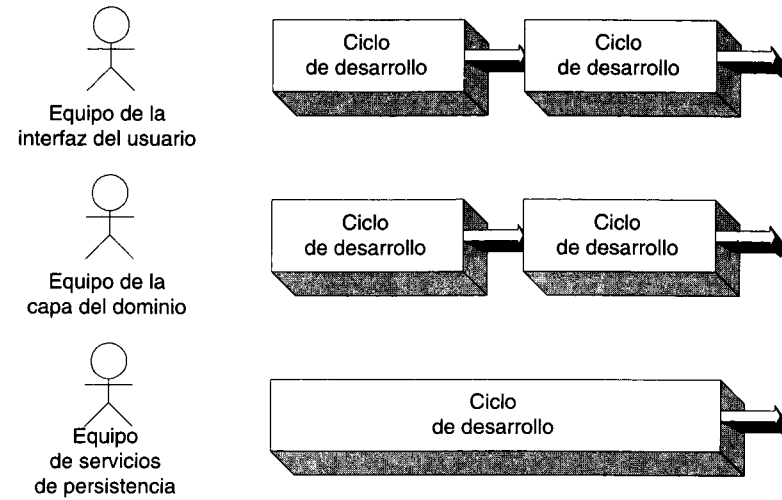


Figura 37.14 Ciclos de desarrollo de duración variable.

37.9.3 Casos de desarrollo de sistemas

Para aclarar los requerimientos técnicos no evidentes de la arquitectura, aconsejamos formular **casos de desarrollo de sistemas**, una especie de casos de uso que describen el trabajo de diseño y las metas funcionales de la actividad en cuestión.¹ Luego pueden programarse los casos como los que están orientados al dominio.

He aquí algunos casos de desarrollo de sistemas:

- Diseñar las capas arquitectónicas y los subsistemas.
- Desarrollar un servicio de persistencia.

De hecho, a todas las actividades relacionadas con el desarrollo y el mantenimiento del software podemos considerarlas casos del desarrollo de sistemas y modelarlas como casos de uso normales de procesos de un dominio, desarrollándoles después diagramas de casos del desarrollo del sistema e indicando, las relaciones *usa*, etcétera (figura 37.5).

En consecuencia, podemos considerar un caso de desarrollo de sistemas, como un caso de uso, y asignarlo a los ciclos de desarrollo.

¹ Se relacionan con los bloques de procesos, de procedimientos y actividades en el modelo de casos de desarrollo propuesto por Jacobson [Jacobson92].

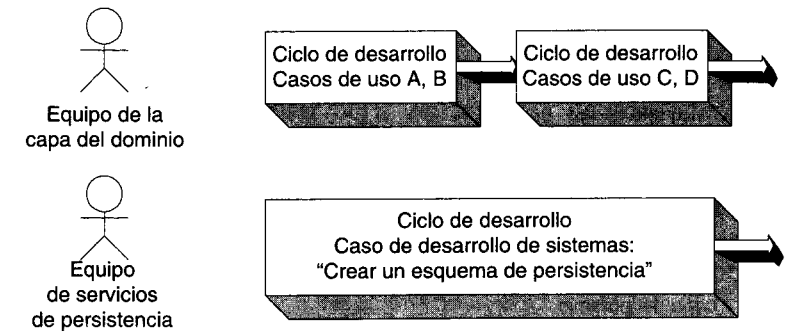


Figura 37.15 Asignación de un caso de desarrollo de sistemas a un ciclo de desarrollo.

37.9.4 Dependencias del desarrollo en paralelo

Cuando varios equipos trabajan en un proyecto, se dan dependencias entre ellos, a saber:

- La interfaz del usuario se basa en la capa de dominio.
- La capa de dominio se basa en las de servicios, como la persistencia.
- El equipo de pruebas (si existe) necesita que algo haya sido terminado y sea lo bastante estable para poder probarlo.

En esta situación el problema radica en que un equipo quiere utilizar el resultado proveniente de otro, aunque no esté terminado. Entre las estrategias para manejar estas dependencias, figuran las siguientes:

- *Talones*: las llamadas a servicios incompletos se implementan como talones de hacer nada o hacer lo mínimo posible. Al irse realizando los servicios, se remplazan los talones.
- *Ciclos escalonados de desarrollo*. Algunos equipos operan sobre un ciclo que se relaciona con el trabajo terminado de un ciclo anterior de otro equipo. Al equipo de la interfaz del usuario le resulta muy cómodo aprovechar los resultados de un ciclo anterior de desarrollo efectuado por un equipo de la capa del dominio (figura 37.16).

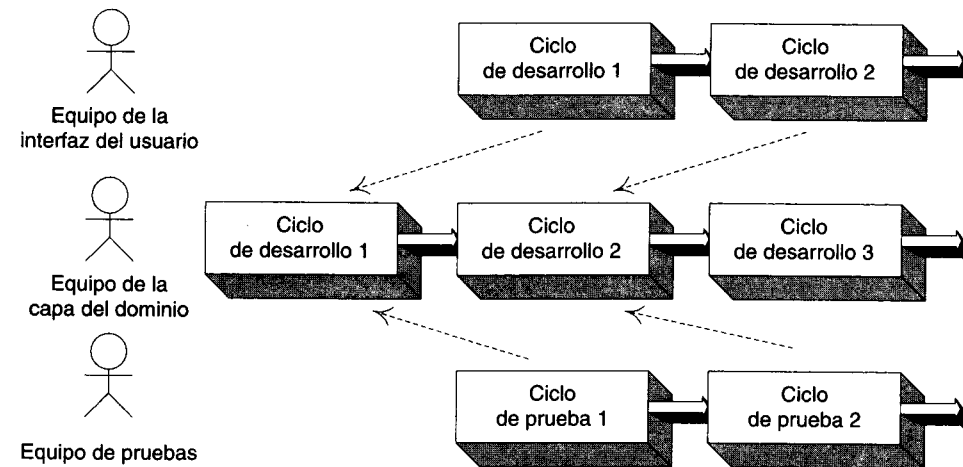


Figura 37.16 Ciclos escalonados dependientes de desarrollo y de pruebas.

37.10 Programación del desarrollo de las capas arquitectónicas

Las capas normales de un sistema son el dominio, el acceso a la base de datos (persistencia), los servicios de alto nivel y la interfaz del usuario. No existe una regla infalible sobre las capas donde hay que centrarse al iniciar el desarrollo, pero sí hay algunas cuestiones a considerar:

- Por lo regular la interfaz del usuario es la concretización visible del sistema. Es lo que ven personas ajenas al desarrollo (directivos y clientes) y que para ellas constituye el sistema. En el ser humano predomina el sentido de la vista. Casi siempre una interfaz atractiva le procurará satisfacción al cliente, aun cuando no sea muy funcional. De ahí la conveniencia de comenzar el proyecto con una interfaz atractiva para el usuario. La desventaja consiste en que los clientes, al verla, creen que el sistema está casi concluido, cuando en realidad apenas si acabamos de iniciarlo.
- En un ciclo de desarrollo, si un equipo está desarrollando la capa del dominio y de los servicios, aconsejamos comenzar con la del dominio. Terminela hasta que se cumplan todas las funciones relacionadas con los casos de la iteración actual. Defina como talones para no hacer nada la capa que brinde soporte a los servicios de alto nivel (los de persistencia o de presentación de reportes, por ejemplo). Por último, implemente estos servicios etiquetados.
- Incorpore el soporte de persistencia (base de datos) al inicio del desarrollo, pero no inmediatamente.
 - Tal vez se requieran demasiados recursos para desarrollar los servicios de persistencia y el soporte a la base de datos. Si abordamos prematuramente esta tarea, tal vez ya no prestemos mucha atención a una capa de dominio bien diseñada. Por desgracia, en la tecnología actual

posponer la tarea suele favorecer el ajuste regresivo o modificaciones de diseño por la interacción de metadatos, el esquema de la base de datos, el diseño de jerarquía de clases, la administración de las transacciones, etcétera.

- Cuando se desarrollan los servicios de persistencia, se genera un esquema mínimo de base de datos y también el marco necesario de persistencia para continuar la iteración actual. Junto con la realización de los procesos de dominio, se mejorarán de modo incremental los servicios de persistencia a lo largo de varios ciclos de desarrollo.

ESQUEMAS, PATRONES Y PERSISTENCIA

Objetivos

- Definir lo que es un esquema (framework).
- Aplicar el patrón más fundamental de esquema de la Pandilla de los Cuatro: el método de plantilla.
- Aplicar otros usos de patrones en los esquemas de persistencia, entre ellos la Instanciación de Objetos Complejos.
- Aplicar otros patrones de la Pandilla de los Cuatro, especialmente Agente Virtual.

38.1 Introducción

En la generalidad de las aplicaciones es necesario guardar y recuperar la información en un mecanismo de almacenamiento persistente, una base de datos relacional por ejemplo. Este capítulo versa sobre un diseño orientado a objetos de un esquema o estructura para objetos persistentes que deben guardarse en un almacenamiento persistente.

No vamos a explicar el diseño de un esquema de persistencia de calidad industrial: el problema de los esquemas de persistencia nos servirá para exponer las cuestiones generales concernientes al diseño de esquemas, así como otros aspectos fundamentales de los servicios de persistencia. También explicaremos la manera de emplear el lenguaje UML para comunicar un diseño.

En este libro no se abordan todos los temas del diseño; las figuras contienen comentarios donde se profundizan aspectos que no podrían expresarse tan claramente por otros medios.

38.2 El problema: objetos persistentes

Supongamos que, en la aplicación del punto de venta, muchas instancias de *EspecificaciondeProducto* residieran en algún mecanismo de almacenamiento persistente —una base de datos por ejemplo—, las cuales han de ser introducidas en la memoria local durante la aplicación.

En la mayoría de las aplicaciones hay que guardar datos y recuperarlos de un mecanismo de almacenamiento persistente, digamos una base de datos relacional u orientada a objetos. Los **objetos persistentes** son aquellos que requieren almacenamiento persistente, entre ellos las instancias *EspecificaciondeProducto*.

El tema de este capítulo es la manera de diseñar servicios de objetos persistentes, porque es un problema común y por ser un instrumento que ayuda a aprender más patrones y los principios del diseño orientado a objetos.

38.2.1 Mecanismos de almacenamiento y los objetos persistentes

Bases de datos orientadas a objetos. Si con una de ellas se almacenan y recuperan objetos, no se necesitan más servicios específicos de persistencia ni de terceros. Éste es uno de los atractivos de su uso.

Bases de datos relacionales. Dado el predominio de este tipo de bases de datos, a menudo se requiere su uso en vez de otras bases más manejables orientadas a objetos. De ser así, surgen varios problemas a causa de la desigualdad entre las representaciones de datos orientadas a registros y las que se orientan a objetos; estos problemas los abordaremos más adelante. Se requieren servicios especiales de ambos tipos en las bases de datos relacionales.

Otros. Además de las bases relacionales de datos, a veces conviene guardar los objetos en otros mecanismos de almacenamiento: archivos planos, bases de datos jerárquicas, etc. Igual que con las bases de datos relacionales, existe divergencia representacional entre las formas orientadas a objetos y el resto de las formas en que se almacenan en esos mecanismos. Como en el caso de las bases de datos relacionales, se necesitan servicios especiales para lograr que funcionen con objetos.

38.3 La solución: un esquema de persistencia

Un **esquema o estructura de persistencia** es un conjunto reutilizable —y, generalmente, expansible— de clases que prestan servicios a los objetos persistentes. Suele escribirse un esquema de persistencia para trabajar con bases de datos relacionales o un API común de servicios de datos orientados a registros; por ejemplo, ODBC de Microsoft. Por lo regular, un esquema de persistencia debe traducir los objetos a registros y guardarlos en una base de datos; después traduce los registros a objetos cuando los recupera de una base (figura 38.1).

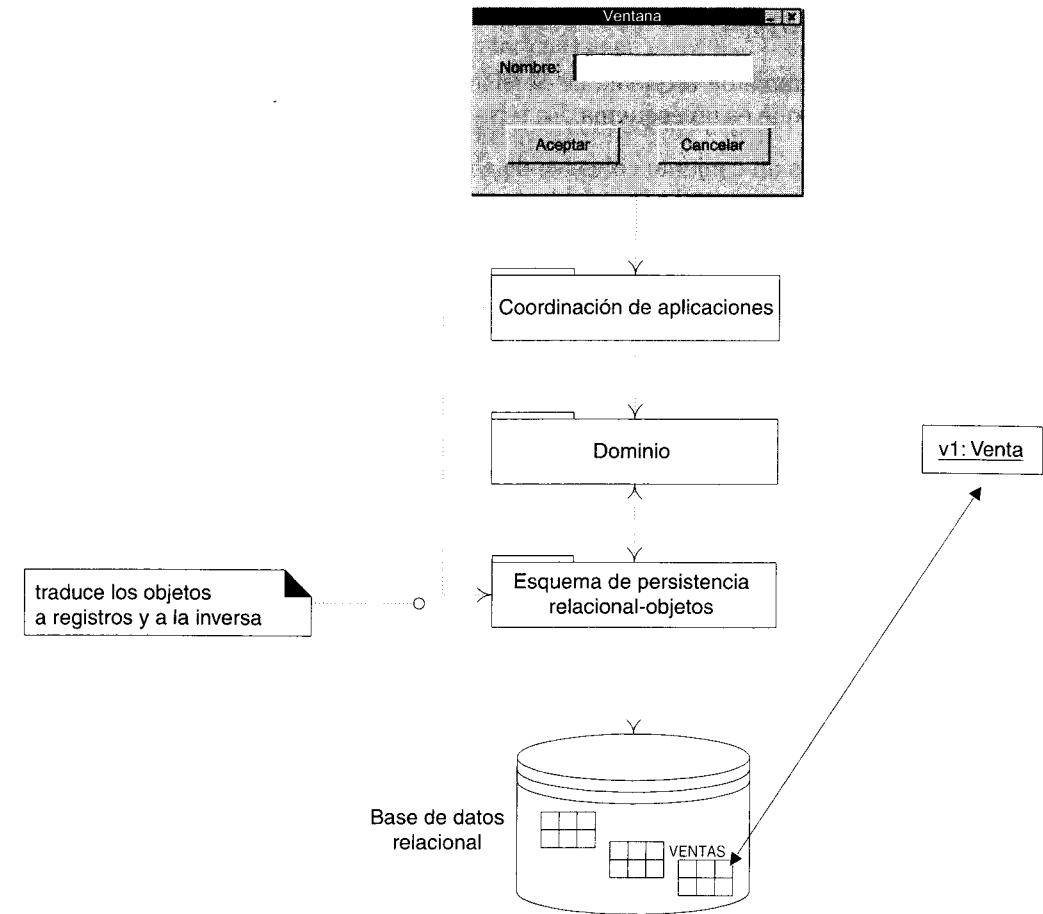


Figura 38.1 Esquema de persistencia relacional-objetos.

Si se emplea una base de datos orientada a objetos, no hace falta un esquema de persistencia. Pero si se utiliza otro mecanismo de almacenamiento, se recomienda servirse de ese esquema; por ejemplo, es de gran utilidad en las bases de datos relacionales y en los archivos planos. En este capítulo vamos a examinar el diseño de un esquema de persistencia y los patrones aplicables a él.

38.4 ¿Qué es un esquema (framework)?

A riesgo de simplificar demasiado el concepto, diremos que un esquema es un subsistema expansible de un conjunto de servicios afines; por ejemplo:

- Los esquemas de interfaz gráfica para el usuario (Foundation Classes de Microsoft, Model-View-Controller de Smalltalk-80).